

Bachelorarbeit

Oleksii Baida
Matrikelnummer 7210384

Entwicklung eines Mikrocontrollersystems für Steuerungs- und
Überwachungsaufgaben innerhalb von Häusern

Bericht

17. April 2025

Inhaltsverzeichnis

Abstract	3
Eigenständigkeitserklärung	4
1 Einleitung	5
2 Ziel des Projekts	6
2.1 Komponenten des Systems	6
2.2 Architektur und Datenfluss	7
2.2.1 Struktur der MQTT-Kommunikation	9
3 Grundlagen & Theorie	10
3.1 Hardware	10
3.1.1 Arduino Uno	10
3.1.2 ESP8266	10
3.1.3 ESP32	11
3.1.4 BME680	12
3.1.5 VCNL4040	12
3.1.6 Raspberry Pi	13
3.2 Kommunikationsprotokolle	14
3.2.1 HTTP	14
3.2.2 MQTT	15
3.2.3 Asynchrone serielle Kommunikation	16
3.2.4 Inter-Integrated Circuit	16
3.3 Software	17
3.3.1 C-Programme	17
3.3.2 Python-Programme	17
3.3.3 Webentwicklung	19
4 Einrichtung der Hardwarekomponenten	22
4.1 Raspberry Pi	22
4.2 Arduino	22
4.3 Einrichtung der ESP-Module	23
4.3.1 ESP8266	28
4.3.2 M5Stick	28
4.3.3 ESP32	29
5 Softwareentwicklung	31
5.1 Datenbank	31
5.2 Webbasierte Schnittstelle (API)	40
5.2.1 Sicherheit	40
5.2.2 API-Endpunkte	42
5.3 Szenario	45
5.4 MQTT-Client	47
5.5 WebSocket	49
5.6 Frontend	50
5.6.1 Dynamische Datendarstellung mit JavaScript	50
5.6.2 Styling	59

6	Ergebnisse und Diskussion	61
6.1	Anwendung des Systems	61
6.2	Vorschläge zur Weiterentwicklung und Erweiterung	64
7	Quellen	65
	Abbildungsverzeichnis	66
	Programmcode	66

Abstract

In dieser Bachelorarbeit wurde ein System zur Überwachung der Umgebungsbedingungen und zur Steuerung der IoT-Geräte im Haus entwickelt. Dabei kamen verschiedene Mikrocontroller, Sensoren und Kommunikationsprotokolle zum Einsatz.

Die Kommunikation zwischen den Geräten erfolgt über das MQTT-Protokoll in einem lokalen Netzwerk ohne Internetverbindung. Zur benutzerfreundlichen Bedienung des Systems wurde eine API mit Webinterface entwickelt.

Der Raspberry Pi dient als zentraler Knoten im System. Er fungiert gleichzeitig als WLAN-Access-Point, MQTT-Broker und Host für den Webserver. Dadurch ist das gesamte System unabhängig von externer Infrastruktur und kann vollständig autonom betrieben werden – auch in Umgebungen ohne Internetzugang.

Die Erweiterung des Systems um zusätzlichen Sensoren und Aktoren ist mit geringem Aufwand möglich. Neue Komponenten können einfach in die bestehende Infrastruktur integriert werden. Der Programmcode der bereits eingesetzten Mikrocontroller enthält viele Standardfunktionen, die modular aufgebaut und leicht wiederverwendbar sind. Dadurch können weitere Komponenten standardmäßig im System eingebunden werden.

Insgesamt zeigt diese Arbeit, wie mit einfachen Mitteln ein robustes und flexibel erweiterbares IoT-System entwickelt werden kann, das sich besonders für den Einsatz in Privathäusern eignet.

Englisch

This bachelor thesis presents the development of a system for monitoring environmental conditions and controlling IoT devices within a house. A variety of microcontrollers, sensors and communication protocols have been used.

The communication between the devices is based on the MQTT protocol within a local network without internet access. An API with a web interface was developed to make the system accessible to users.

The Raspberry Pi is the central node of the system. It simultaneously acts as a Wi-Fi access point, MQTT broker and host for the web server. This setup allows the entire system to be location-independent and run completely autonomously.

The system can easily be expanded with additional sensors and actuators. New components can be integrated into the existing infrastructure with minimal effort. The programme code on the microcontrollers used contains modular standard functions that are easy to reuse. This allows additional devices to be easily integrated into the system.

Overall, this work demonstrates how a robust and flexibly expandable IoT system can be developed using simple components, making it suitable for use in private homes.

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbständig angefertigt und mich keiner fremden Hilfe bedient sowie keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Alle Stellen, die wörtlich oder sinngemäß veröffentlichten oder nicht veröffentlichten Schriften und anderen Quellen entnommen sind, habe ich als solche kenntlich gemacht.

Soweit Sprachmodelle für die Texterstellung (z.B. ChatGPT) verwendet wurden, sind die entsprechenden Textstellen gekennzeichnet. Die Nutzung von Korrekturservices ist ebenfalls kenntlich gemacht.

Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

1 Einleitung

In einer Welt, die zunehmend von vernetzten Geräten und dem Internet der Dinge geprägt ist, wird die Entwicklung effizienter und benutzerfreundlicher Systeme zur Steuerung und Überwachung von Gebäuden immer relevanter. Im Jahr 2024 wurde in Deutschland die Anzahl von über 19 Millionen Haushalten, die ein oder mehrere smarte Geräte besaßen, verzeichnet. Prognosen zufolge wird sich diese Zahl in den nächsten drei Jahren verdoppeln [3].

Moderne Steuerungs- und Überwachungssysteme tragen zur Effizienzsteigerung und Ressourcenschonung bei. Diese Systeme übernehmen einen Teil der täglichen Aufgaben, wie das Ein- und Ausschalten des Lichts, die Regelung der Raumtemperatur oder das Aufräumen des Hauses. Die möglichen Funktionen sind dabei nur durch die Bedürfnisse der Benutzerinnen und Benutzer begrenzt.

Im Rahmen meiner Bachelorarbeit wird ein System zur Steuerung und Überwachung des Hauses entwickelt. Es wird eine Web-Schnittstelle erstellt, die die Interaktion des Benutzers mit den Geräten in seinem Haushalt ermöglicht. Über diese Schnittstelle werden die von Geräten erfassten Daten dargestellt und die Benutzer werden über gefährliche Vorgänge im Haus informiert. Außerdem haben die Benutzer die Möglichkeit eigene Szenarien basierend auf den erfassten Daten zu erstellen.

Zur Erfassung und Verarbeitung der Sensordaten werden Mikrocontroller eingesetzt. Diese sind entsprechend programmiert, um die Sensordaten zu erfassen und die Daten über das Netzwerk an den zentralen Server zu übermitteln.

Im Rahmen der Entwicklung dieses Systems wurden die aktuellen Technologien zur Erstellung eines Webinterfaces und zur Kommunikation zwischen den Geräten eingesetzt. Das System bleibt dabei flexibel und erweiterbar.

Die Bachelorarbeit gliedert sich in mehrere Kapitel. Kapitel 2 behandelt die Ziele des Projekts sowie die allgemeine Systemarchitektur und die einzelnen Komponenten des Systems.

In Kapitel 3 werden die theoretischen Grundlagen zur Entwicklung des Systems sowie die eingesetzten Technologien, Hardwarekomponenten und Protokolle erläutert. Kapitel 4 beschreibt die Einrichtung der im Projekt verwendeten Geräte.

In Kapitel 5 werden die Softwarekomponenten des Systems beschrieben. Die Entwicklung des Back- und Frontends sowie die Verarbeitung der im System laufenden MQTT-Nachrichten werden detailliert dargestellt.

In Kapitel 6 wird ein konkreter Anwendungsfall des Systems beschrieben. Daneben werden mögliche Weiterentwicklungen und Optimierungspotenziale aufgezeigt, um das System in Zukunft zu erweitern und zu verbessern.

2 Ziel des Projekts

Das im Rahmen dieser Arbeit entwickelte System stellt einen Prototyp für ein Smart-Home-System dar. Ziel des Projekts ist die Realisierung eines intelligenten Heimsystems, das verschiedene Technologien integriert und flexibel erweiterbar ist. Die Architektur des Systems ist modular aufgebaut, sodass die Geräte hinzugefügt und eingebunden werden können. Dies erlaubt eine skalierbare und anpassbare Nutzung in unterschiedlichen Anwendungsfällen.

Das entwickelte System dient der Überwachung und Steuerung von vernetzten Geräten innerhalb eines Hauses. Ziel des Systems ist es, Sensordaten zu erfassen, diese zu verarbeiten und eine benutzerfreundliche Interaktionsmöglichkeit zu bieten. Dabei kommen verschiedene Hardware- und Softwarekomponenten zum Einsatz, die eine nahtlose Kommunikation zwischen den einzelnen Modulen gewährleisten.

Das System erfüllt mehrere wesentliche Aufgaben:

1. **Sensordatenerfassung:** Die an den Mikrocontrollern angeschlossenen Sensoren übermitteln die erfassten Daten an dem zentralen Server.
2. **Sicherheitsaspekt:** Das System ist mit den Sensoren für die Erkennung des Brandes ausgestattet. Außerdem bietet das System gesicherter Zugang zu dem Haus.
3. **Datenverarbeitung und Speicherung:** Der zentrale Server stellt die empfangenen Daten zur Analyse bereit und speichert die Benutzer- und Geräteinformationen in der Datenbank.
4. **Benutzerinteraktion:** Über die Weboberfläche können die Nutzer das System konfigurieren, Messwerte einsehen und Aktoren steuern. Außerdem werden die Nutzer über das Brand im Haus informiert.
5. **Kommunikation:** Die Kommunikation zwischen den Komponenten erfolgt über das MQTT-Protokoll, wodurch eine effiziente und skalierbare Datenübertragung ermöglicht wird.
6. **Automatisierung:** Die Benutzer können eigene Szenarien erstellen, die auf Basis der erfassten Sensordaten automatisch ausgelöst werden.

2.1 Komponenten des Systems

Das System besteht aus mehreren Komponenten, die zusammenarbeiten, um eine effiziente Steuerung und Datenerfassung zu gewährleisten:

- **Raspberry Pi:** Der zentrale Server. Stellt das lokale WLAN-Netzwerk bereit, hostet den MQTT-Broker und führt die Webanwendung aus.
- **Arduino:** Ausgestattet mit mehreren Sensoren, die Gefahren wie Feuer und Gas erkennen und den Zugang zum Haus sichern. Entsprechende Meldungen werden an den Server gesendet.
- **M5Stick:** Angeschlossen an die Temperatur- und Lichtsensoren und sendet die aufgezeichneten Daten auf den Server.
- **ESP32:** Angeschlossen an Display und Temperatursensor dient zur Anzeige der Umgebungsdaten und sendet diese an den Server.

- **Webserver:** Eine auf FastAPI basierende RESTful API, die Benutzern den Zugriff auf das System und die Steuerung von Geräten ermöglicht.
- **Datenbank:** Eine lokale Datenbank zur Speicherung von Benutzerinformationen und Gerätekonfigurationen.
- **Web-Anwendung:** Eine browserbasierte Benutzeroberfläche, die den Benutzern eine intuitive Steuerung und Visualisierung der Daten ermöglicht.

2.2 Architektur und Datenfluss

Das entwickelte System basiert auf einer modularen Architektur, die mehrere Komponenten integriert. Die Abbildung `abb:gesamtsystem` stellt das gesamte System dar.

Der Raspberry Pi dient als zentraler Server des Systems. Der Minicomputer stellt ein WLAN-Netzwerk zur Verfügung und hostet den Webserver sowie den MQTT-Broker. Alle Benutzerinteraktionen, Sensordaten, Datenflüsse und Datenverarbeitungen werden auf dem Raspberry Pi durchgeführt. Das System ist somit stark zentralisiert und arbeitet nur in einem lokalen Netzwerk. Daher müssen sich alle Benutzer mit dem vom Raspberry Pi bereitgestellten WLAN-Netzwerk verbinden.

Die Geräte verbinden sich mit dem WLAN und mit dem MQTT-Broker, die vom Raspberry Pi zur Verfügung gestellt werden. Jedes Gerät verfügt über eine eindeutige Geräte-ID. Die Sensoren senden ihre Daten an den MQTT-Broker, der auf dem Raspberry Pi läuft. Die Aktoren abonnieren die Command-Topics, die für jedes Gerät mit der entsprechenden Geräte-ID erstellt werden.

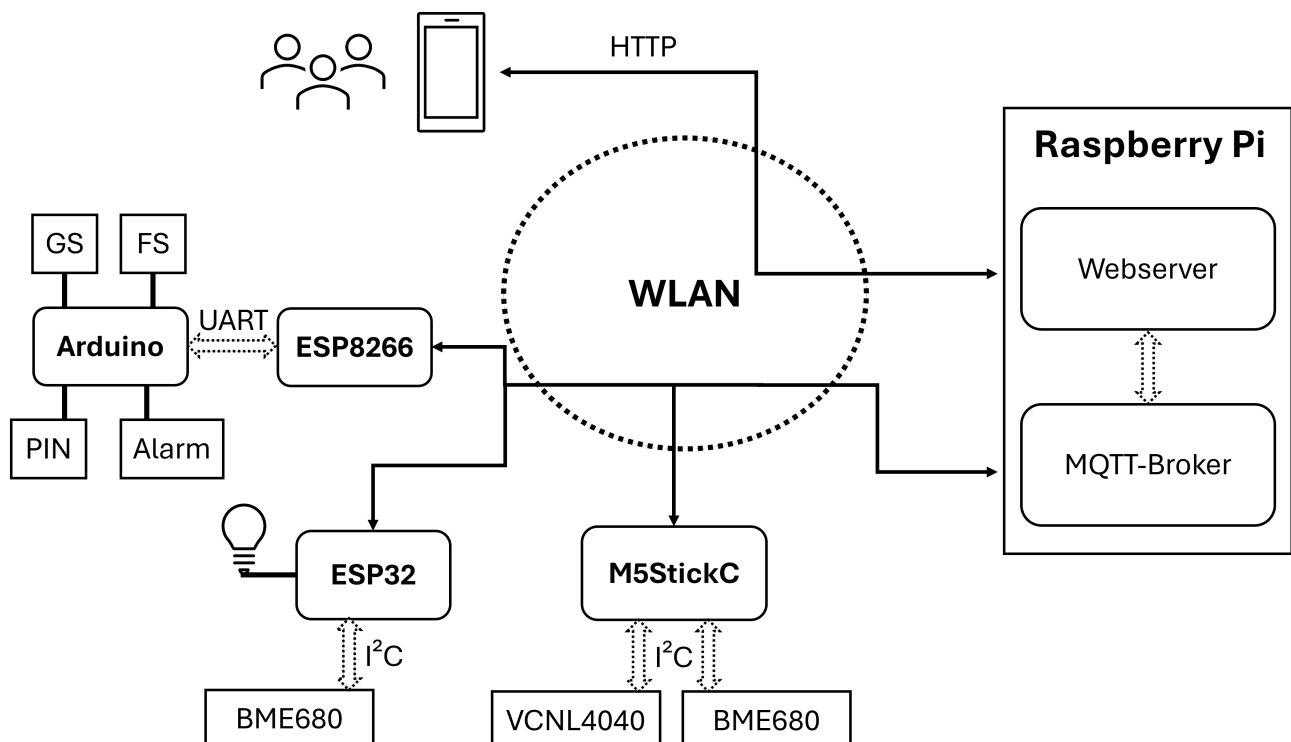


Abbildung 1: Gesamtsystem

Die Schnittstelle zu den Benutzer wurde über einen Webserver realisiert. Der Webserver stellt eine RESTful-API zur Verfügung. Für die Kommunikation mit der API wurde eine Weboberfläche entwickelt, die den Benutzern zur Verfügung steht. Die Benutzer können auf das System ausschließlich über den Webserver zugreifen. Die API bietet folgende Funktionen an:

- Authentifizierung und Autorisierung der Benutzer.
- Verwaltung der Gerätekonfigurationen und Benutzerprofile.
- Bereitstellung von Endpunkten zur Abfrage und Steuerung von IoT-Geräten.
- Bereitstellung der Daten von den Geräten in Echtzeit.
- Bidirektionale Kommunikation mit den IoT-Geräten.
- Erstellung der Automatisierungsszenarien.

Die Benutzerauthentifizierung erfolgt über JSON Web Tokens (JWT). Bei der Anmeldung wird ein JWT erzeugt, das eine verschlüsselte Benutzer-ID enthält. Dieses Token wird auf der Seite des Benutzers gespeichert und bei jeder Anfrage an den Server gesendet, um die Identität des Benutzers zu verifizieren. JWTs haben eine festgelegte Lebensdauer, nach deren Ablauf eine erneute Anmeldung oder eine Token-Erneuerung erforderlich ist. Die Verwendung von JWT ermöglicht eine sichere, zustandslose Authentifizierung, da der Server keine Sitzungsdaten speichern muss.

Die Benutzerdaten und Gerätekonfigurationen werden in einer lokalen Datenbank gespeichert, die mit `SQLite` implementiert wird. `SQLite` hat den Vorteil, dass die gesamte Datenbank in einer einzigen Datei gespeichert wird, wodurch der Ressourcenverbrauch des Servers minimiert wird. Der Zugriff auf die Datenbank erfolgt über `SQLAlchemy`, eine leistungsfähige ORM-Bibliothek für Python. `SQLAlchemy` ermöglicht asynchrone Datenbankzugriffe, was die Leistung und Skalierbarkeit der Anwendung verbessert.

Die Geräte senden ihre Daten an den zentralen MQTT-Broker. Um die eingehenden Nachrichten effizient verarbeiten zu können, wird ein MQTT-Client mit Hilfe der Bibliothek `aiomqtt` implementiert. Diese Bibliothek verwendet Python `asyncio`, was eine nicht-blockierende, asynchrone Verarbeitung ermöglicht. Dadurch können Nachrichten empfangen und verarbeitet werden, ohne den Hauptprozess zu blockieren. Das verbessert die Reaktionsfähigkeit des Systems und ermöglicht die parallele Verarbeitung mehrerer MQTT-Nachrichten, was besonders wichtig für die Systeme mit mehreren Geräten ist.

Die Gerätedaten müssen den Benutzern in Echtzeit zur Verfügung gestellt werden. Zu diesem Zweck wird `WebSocket` verwendet. Es ermöglicht eine kontinuierliche und bidirektionale Datenübertragung zwischen Server und Client. Dadurch wird sichergestellt, dass die Daten sofort und ohne wiederholte Anfragen an den Benutzer übermittelt werden.

Der Benutzer benötigt eine Schnittstelle, um auf das System zugreifen zu können. Dazu wird eine Webseite in Form einer SPA¹ entwickelt. Eine SPA lädt einmal die grundlegende HTML-, CSS- und JavaScript-Struktur und aktualisiert dann dynamisch nur die relevanten Inhalte, ohne die gesamte Seite neu zu laden. Der Hauptvorteil einer SPA liegt in der verbesserten Nutzererfahrung, da Seitenwechsel nahezu verzögerungsfrei erfolgen. Außerdem wird die Serverlast reduziert, da nur die benötigten Daten über eine API geladen werden, anstatt komplette HTML-Dokumente zu übertragen. Dies führt zu einer schnelleren und reaktionsfähigeren Anwendung, was insbesondere bei Anwendungen mit Echtzeitanforderungen von Vorteil ist.

Die für die SPA entwickelte API kann auch von anderen Anwendungen, z. B. mobilen Anwendungen, genutzt werden. Dadurch können die Anwendungen auf einer gemeinsamen Backend-Struktur aufbauen, was die Wartung vereinfacht und die Wiederverwendbarkeit des Codes erhöht.

Die vorgestellte Architektur bietet eine effiziente, skalierbare und reaktionsfähige Struktur. Der Zugriff auf das System erfolgt über die API. Durch die Verwendung asynchroner Methoden

¹Engl. Single Page Application

werden Blockierungen vermieden. **WebSocket** stellt die empfangenen Daten ohne serverseitige Speicherung sofort zur Verfügung. Die SPA bietet eine schnelle und benutzerfreundliche Schnittstelle mit direktem Zugriff auf die API-Endpunkte. Diese Architektur garantiert eine hohe Performance und vereinfacht sowohl die Wartung als auch zukünftige Erweiterungen des Systems.

2.2.1 Struktur der MQTT-Kommunikation

Die im System verwendeten Geräte kommunizieren über das MQTT-Protokoll. Der MQTT-Broker wird auf dem Raspberry Pi ausgeführt.

Die Geräte verbinden sich mit dem MQTT-Broker und senden ihre Nachrichten in individuelle Topics. Es gibt 4 Haupttopics, die für jedes Gerät mit der entsprechenden Geräte-ID erweitert werden. Die Tabelle 1 listet die Topics auf. ID wird von jedem Gerät ermittelt und beim Senden der Nachricht entsprechend gesetzt.

Topic	Nachricht	Beschreibung
handshake/ID	<code>"data_fields": ["temperature", "humidity"], "commands": ["light_on", "light_off"]</code>	In diesem Topic senden die Geräte ihre Konfigurationen.
data/ID	<code>"temperature": 23, "humidity": 75</code>	Hier werden die von den Sensoren erfassten Daten übertragen.
command/ID	<code>"light_on"</code>	Dieses Topic wird von Geräten abonniert. Der in der Nachricht übermittelte Befehl wird vom Gerät ausgeführt.
alarm/ID	<code>"fire": "fire_start"</code>	Dies ist ein spezielles Topic für das Alarmsystem. Wenn der Alarm ausgelöst wird, wird die entsprechende Meldung in diesem Topic gesendet.

Tabelle 1: MQTT-Nachrichten

Beim Hinzufügen eines neuen Geräts zum System sendet der dem Broker eine Handshake-Nachricht in das Topic „**command/ID**“ mit dem Text „**handshake**“ gesendet. Das Gerät antwortet darauf im Topic „**handshake/ID**“ mit seiner Konfiguration in JSON-Format. Im Feld „**data_fields**“ werden die Felder angegeben, die von den Sensoren erfasst werden. Im Feld „**commands**“ werden die Befehle angegeben, die das Gerät ausführen kann. Diese Daten werden in der Datenbank gespeichert, können jedoch durch erneutes Senden der Handshake-Nachricht aktualisiert werden. Nach der Handshake-Nachricht werden in das Topic „**command/ID**“ anschließend Nachrichten mit gespeicherten Befehlen gesendet.

Die Nachrichten, die in das Topic „**data/ID**“ gesendet werden, müssen ein JSON-Format haben. Das erleichtert die Verarbeitung der Daten.

3 Grundlagen & Theorie

In diesem Abschnitt werden die technischen Spezifikationen der verwendeten Komponenten und Technologien detailliert beschrieben.

3.1 Hardware

Das Projekt umfasst verschiedene Hardware-Komponenten, darunter Mikrocontroller, Sensoren und einen Einplatinencomputer.

3.1.1 Arduino Uno

Der Arduino Uno ist ein Mikrocontroller-Board, das sich besonders für die Erfassung und Verarbeitung von Sensordaten eignet. Aufgrund seiner einfachen Programmierbarkeit und vielseitigen Schnittstellen wird es häufig in eingebetteten Systemen sowie in IoT-Anwendungen eingesetzt.

Das Herz des Boards bildet der Mikrocontroller ATmega328P, der über Flash-Speicher und EEPROM² verfügt. Das EEPROM ermöglicht die nichtflüchtige Speicherung von Daten, so dass diese auch nach dem Ausschalten des Gerätes erhalten bleiben.

Die technischen Spezifikationen des Arduino Uno sind in Tabelle 2 dargestellt.

Modell	Arduino Uno
Mikrocontroller	ATmega328P
Taktfrequenz	16 MHz
Eingangsspannung	12 V
Digital Pins	14
Analoge Eingänge	6
Flash-Speicher	32 KB
SRAM	2 KB
EEPROM	1 KB
Kommunikationsschnittstellen	UART, SPI, I2C
USB-Schnittstelle	USB-B

Tabelle 2: Technische Daten des Arduino Uno

Im Rahmen dieses Projekts wird der Arduino Uno zur Erfassung und Verarbeitung von Sensordaten für die Branderkennung eingesetzt. Darüber hinaus übernimmt er die Steuerung des kontrollierten Zugangs zum Gebäude. Durch die Anbindung an verschiedene Sensoren ermöglicht er eine kontinuierliche Überwachung der Umgebung und kann im Gefahrenfall entsprechende Meldungen auslösen.

3.1.2 ESP8266

Das ESP8266 ist ein kompakter Mikrocontroller mit integriertem WLAN-Modul. Aufgrund seiner kompakten Größe, seines geringen Stromverbrauchs und seiner integrierten WLAN-Schnittstelle eignet er sich besonders für drahtlose Sensornetzwerke und eingebettete Systeme.

²Electrically Erasable Programmable Read-Only Memory

Ein zentrales Funktionsmerkmal des ESP8266 ist seine integrierte WLAN-Funktionalität. Das Modul kann direkt mit einem WLAN-Netzwerk verbunden werden oder als Access Point ein WLAN-Netzwerk zur Verfügung stellen. Zusätzlich kann auf dem Mikrocontroller ein einfacher Webserver eingerichtet werden. Damit kann es sowohl als eigenständiger Mikrocontroller als auch als Wi-Fi-Erweiterung für andere Mikrocontroller, z.B. einen Arduino, verwendet werden.

Das Modul basiert auf dem ESP8266 Chip von Espressif Systems. Es verfügt über Flash, SRAM und EEPROM Speicher. Außerdem unterstützt es das UART-Kommunikationsprotokoll.

Die technischen Spezifikationen des ESP8266-01 sind in der Tabelle 3 dargestellt.

Modell	ESP8266-01
Chip	ESP8266EX
Architektur	32-Bit
Taktfrequenz	80 MHz
Eingangsspannung	5 V
Flash-Speicher	1 MB
SRAM	80 KB
Kommunikationsschnittstellen	UART
WLAN-Standard	IEEE 802.11
Sicherheitsmechanismen	WEP, WPA, WPA2

Tabelle 3: Technische Daten des ESP8266

In meinem Projekt wird das ESP8266 als Schnittstelle zwischen dem Arduino und dem zentralen Server verwendet. Er ermöglicht die drahtlose Übertragung von Sensordaten an das Backend und kann auch Steuerbefehle empfangen, um entsprechende Aktionen auszulösen. Durch die direkte WLAN-Anbindung trägt das ESP8266 wesentlich zur flexiblen und ortsunabhängigen Systemsteuerung bei.

3.1.3 ESP32

Das ESP32 ist ein leistungsstarker Mikrocontroller von Espressif Systems. In der Literatur wird das Modul auch als SoC³ bezeichnet, was bedeutet, dass alle Funktionen des Moduls auf einem Chip integriert sind.

Das ESP32 unterstützt WLAN und Bluetooth. Außerdem verfügt es über Schnittstellen wie I2C, UART, GPIOs, die eine flexible Integration mit Sensoren und anderen Peripheriegeräten ermöglichen.

Die technischen Spezifikationen des ESP32 sind in der Tabelle 4 dargestellt.

M5StickC

Der M5StickC ist eine kompakte Entwicklungsplattform, die auf dem ESP32 basiert. Er verfügt über ein integriertes Display, eine Batterie und zusätzliche integrierte Sensoren.

Der M5StickC verfügt über ein 1,14-Zoll großes LCD-Display, das die visuelle Ausgabe von Sensordaten oder Statusmeldungen ermöglicht. Außerdem enthält er eine integrierte IMU (Inertial Measurement Unit) zur Bewegungs- und Lageerkennung. Eine integrierte Lipo-Batterie ermöglicht den kabellosen Betrieb über einen längeren Zeitraum.

³Eng. System-on-a-Chip, Deutsch: Ein-Chip-System

Modell	ESP32-WROOM-32
Architektur	32-Bit Dual-Core
Taktfrequenz	Bis zu 240 MHz
Betriebsspannung	3,3 V
Digitale GPIO-Pins	34
Analoge Eingänge	18
Flash-Speicher	4 MB
SRAM	520 KB
Kommunikationsschnittstellen	UART, SPI, I2C, CAN, I2S
USB-Schnittstelle	Mikro-USB
WLAN-Standard	IEEE 802.11 b/g/n
Bluetooth	BLE 4.2, Classic
Energiesparmodi	Deep-Sleep, Light-Sleep

Tabelle 4: Technische Daten des ESP32

Aufgrund seiner kompakten Größe und der langen Batterielaufzeit eignet er sich besonders für IoT-Systeme.

In diesem Projekt dient der M5StickC als tragbare Steuerungs- und Überwachungseinheit. An das Modul werden Temperatur- und Lichtsensoren angeschlossen. Das Modul wird über WLAN mit dem Server verbunden. Auf dem integrierten Display werden die Daten der Sensoren sowie Verbindungszustände und Meldungen angezeigt.

3.1.4 BME680

Der BME680 ist ein kompakter Umweltsensor von Bosh, der mehrere Funktionen in einem Modul vereint. Das Modul ist mit einem Gassensor, einem Feuchtesensor, einem Luftdrucksensor und einem Temperatursensor ausgestattet. Aufgrund seiner kompakten Größe und hohen Genauigkeit wird das Modul häufig im Bereich der Umweltsensorik eingesetzt.

Mit Hilfe des eingebauten Gassensors und einer vom Hersteller speziell entwickelten Software kann die Luftqualität (IAQ - Eng. Index of Air Quality) gemessen werden. Grundsätzlich handelt es sich um eine Zahl von 0 bis 500, wobei 0 die beste und 500 die schlechteste Luftqualität darstellt. Eine detaillierte Beschreibung des Sensors sowie der Bestimmung der IAQ ist im Datenblatt zum Modul BME680 [8] unter Kapitel 1.2 zu finden. Die Tabelle 5 enthält die wichtigsten technischen Daten des Moduls.

Der BME680 verfügt über I2C- und SPI- Schnittstellen für die Kommunikation mit anderen Geräten.

In meinem Projekt wird das Modul BME680 zur Messung von Temperatur und Luftqualität eingesetzt.

3.1.5 VCNL4040

Der VCNL4040 ist ein hochpräzises optisches Modul. Das Modul enthält einen Näherungssensor (Proximity Sensor PR), einen Lichtsensor (Ambient Light Sensor) und eine Infrarot-Leuchtdiode

Modell	BME680
Versorgungsspannung	3,3 V
Schnittstellen	I2C, SPI
Temperaturbereich	-40 bis +85 °C
Luftfeuchtigkeitsbereich	0 – 100 %
Druckbereich	300 – 1100 hPa
Luftqualität	0 - Gute Qualität 500 - schlechte Qualität

Tabelle 5: Technische Daten des BME680

(IRED).

Mit Hilfe des Näherungssensors und der IRED kann das Modul den Abstand zu einem Objekt bis zu einer Entfernung von 200 mm in Echtzeit messen. Der Lichtsensor misst die Helligkeit der Umgebung.

Das VCNL4040 verfügt über eine I2C-Schnittstelle zur Kommunikation mit anderen Geräten.

Eine detaillierte Beschreibung des Moduls sowie die technischen Daten sind dem Datenblatt [9] zu entnehmen. In der Tabelle 6 sind die wichtigsten Daten aufgelistet.

Modell	VCNL4040
Messprinzip	Infrarot-Näherungssensor
Versorgungsspannung	3,3 V
Schnittstellen	I2C
Erfassungsbereich	0 – 200 mm
Maximale Abtastrate	20 Hz

Tabelle 6: Technische Daten des VCNL4040

In diesem Projekt wird das Modul als Lichtsensor zur Bestimmung der Umgebungshelligkeit eingesetzt.

3.1.6 Raspberry Pi

Der Raspberry Pi [6] ist ein Einplatinencomputer, der von der Raspberry Pi foundation entwickelt wurde. Dieser Minicomputer verfügt über einen leistungsfähigen ARM-Prozessor. Dank seiner geringen Größe und hohen Rechenleistung wird er häufig in IoT-Systemen eingesetzt.

Die technischen Daten sind in der Tabelle 7 gelistet.

In diesem Projekt wird der Raspberry Pi als zentraler Server eingesetzt. Er übernimmt die Kommunikation mit den angeschlossenen Sensoren und Aktoren über MQTT und stellt eine Webschnittstelle für Benutzerinteraktionen bereit. Zusätzlich wird er als WLAN-Access-Point konfiguriert, um eine lokale Netzwerkverbindung für die angebundenen IoT-Geräte bereitzustellen.

Modell	Raspberry Pi 1.0 Modell B+
CPU	ARM-Prozessor
Arbeitsspeicher (RAM)	512 MB
USB-Ports	4
MicroSD-Kartensteckplatz	1
HDMI-Anschluss	1
Ethernet-Anschluss	1
GPIO-Pins	40
Wi-Fi	Vorhanden
Bluetooth	Vorhanden
Stromversorgung	5V Micro-USB-Anschluss
Betriebssystem	Raspberry Pi OS Debian 11 „Bullseye“

Tabelle 7: Technische Daten des Raspberry Pi

3.2 Kommunikationsprotokolle

3.2.1 Hypertext Transfer Protocol

Das Hypertext Transfer Protocol (HTTP) ist ein anwendungsorientiertes Kommunikationsprotokoll, das den Datenaustausch zwischen Webservern und Clients ermöglicht. Es dient der Übertragung von Hypertext-Dokumenten zwischen Webservern und Clients, typischerweise Webbrowsern.

HTTP arbeitet nach dem Request-Response-Prinzip, bei dem ein Client eine Anfrage (Request) an einen Server sendet, der daraufhin eine Antwort (Response) zurücksendet. Die Kommunikation erfolgt in der Regel über das Transmission Control Protocol (TCP).

Ein wesentliches Merkmal von HTTP ist seine Zustandslosigkeit (Statelessness). Jede Anfrage wird unabhängig von früheren Anfragen behandelt, so dass das Protokoll keine Informationen über frühere Interaktionen speichert. Um dennoch Sitzungsdaten zu verwalten, werden Mechanismen wie Cookies, Session-IDs oder Token-basierte Authentifizierung verwendet. In diesem Projekt wird JWT (JSON Web Token) für die Benutzerauthentifizierung verwendet.

HTTP definiert mehrere Methoden zur Kommunikation mit dem Webserver, darunter:

- **GET:** Zum Abrufen von Informationen aus dem System.
- **POST:** Zum Übermitteln neuer Daten oder Benutzerinteraktionen.
- **PUT:** Zur Aktualisierung vorhandener Informationen.
- **DELETE:** Zum Löschen von Daten.

Ein HTTP-Request besteht aus mehreren Komponenten:

- **Request-Line:** Enthält die HTTP-Methode, die Ziel-URL und die verwendete HTTP-Version.

- **Header:** Enthält zusätzliche Metadaten wie akzeptierte Datenformate oder Authentifizierungsinformationen.
- **Body:** Enthält bei Methoden wie POST oder PUT die zu übertragenden Daten.

Ein Beispiel für eine HTTP-POST-Anfrage, bei der die Anmeldedaten gesendet werden:

```
POST /login HTTP/1.1
Host: 127.0.0.1
Content-Type: application/json
Content-Length: 32

username=testuser&password=1234
```

Der Server antwortet mit einer HTTP-Response, die ähnlich strukturiert ist. Die erste Zeile der Antwort wird als Statuszeile bezeichnet und enthält die HTTP-Version, den Statuscode und eine Statusbeschreibung.

In diesem Projekt wird HTTP für die Kommunikation zwischen Webserver und Browser verwendet.

3.2.2 MQTT

Das Message Queuing Telemetry Transport (MQTT)-Protokoll [11] ist ein leichtgewichtiges und effizientes Nachrichtenprotokoll, das speziell für den Einsatz in IoT-Systemen entwickelt wurde. Es ermöglicht die asynchrone Kommunikation zwischen Geräten über Netzwerke mit geringer Bandbreite und hoher Latenz.

MQTT basiert auf dem Publish-Subscribe-Modell, das eine zentrale Instanz, den Broker, erfordert. Ein Publisher sendet Nachrichten zu einem bestimmten Topic, während ein Subscriber die Nachrichten eines abonnierten Topics empfängt. Der Broker übernimmt dabei die Filterung, Verteilung und Verwaltung der Nachrichten.

Ein Topic ist ein hierarchisch strukturierter String, die zur Gliederung von Nachrichten dient. Die einzelnen Ebenen eines Topics werden durch einen Slash („/“) getrennt. Dies ermöglicht eine feine Filterung der Nachrichten. Ein Broker verlangt keine vorherige Registrierung von Topics, sondern akzeptiert alle syntaktisch gültigen Topics.

Eine Nachricht ist ein String, der in ein Topic gesendet wird. In vielen Anwendungsfällen werden Nachrichten im JSON-Format übertragen, um eine strukturierte Verarbeitung zu ermöglichen.

MQTT bietet die Möglichkeit, die zuletzt gesendete Nachricht eines Topics zu speichern. Ein neu verbundener Client, der ein solches Topic abonniert, erhält automatisch die letzte gespeicherte Nachricht.

Beim Aufbau einer MQTT-Verbindung sendet der Client eine eindeutige Client-ID an den Broker. Optional kann eine Authentifizierung mittels Benutzername und Passwort erfolgen, um unberechtigten Zugriff auf das System zu verhindern.

Eine weitere sicherheitsrelevante Funktion ist das Last-Will-and-Testament (LWT), das es dem Broker ermöglicht, im Falle eines Geräteausfalls eine bestimmte Nachricht zu senden. Das Testament wird vom Client beim Verbindungsaufbau mit Topic und Message angegeben und vom Broker gespeichert. Zusätzlich wird mit dem Parameter `keepAlive` die maximale Reaktionszeit des Clients festgelegt. Wird diese Zeit überschritten, interpretiert der Broker dies als Ausfall und sendet die hinterlegte LWT-Nachricht.

Aufgrund seiner geringen Ressourcenanforderungen, einfachen Implementierung und Zuverlässigkeit findet MQTT breite Anwendung in Smart-Home-Systemen, industriellen IoT-Lösungen und Telemetrie-Anwendungen. Die Nachrichtenübertragung erfolgt in einer festgelegten Reihenfolge, wodurch eine konsistente Kommunikation gewährleistet wird.

3.2.3 Asynchrone serielle Kommunikation

In dieser Arbeit erfolgt die Kommunikation zwischen Arduino und ESP8266 über eine serielle asynchrone Schnittstelle. Die Daten werden zwischen den Komponenten bidirektional übertragen.

UART⁴ ist eine Hardwarekomponente, die die serielle Datenübertragung bei der Kommunikation zwischen Mikrocontrollern und anderen Geräten ermöglicht. Eine UART-Schnittstelle ist der Standard für serielle Schnittstellen an PCs und Mikrocontrollern und wird zum Senden und Empfangen von Daten über eine Datenleitung verwendet.

UART arbeitet asynchron, d.h. es gibt keine gemeinsame Taktverbindung zwischen den kommunizierenden Geräten. Stattdessen wird die Datenübertragung durch Start- und Stoppbits synchronisiert. Diese ermöglichen es dem Empfänger, den Beginn und das Ende eines Datenpakets zu erkennen. Ein typisches Datenpaket besteht aus einem Startbit, gefolgt von einer festgelegten Anzahl von Datenbits (normalerweise 8), einem optionalen Paritätsbit zur Fehlerkontrolle und einem oder mehreren Stoppbits. Der Vorteil besteht darin, dass keine permanente Synchronisation zwischen Empfänger und Sender erforderlich ist. Sie müssen nur für die Dauer der Übertragung synchronisiert sein. Wenn keine Daten zu übertragen sind, setzt der Sender die Leitung auf die Polarität des Stoppbits.

Ein Vorteil der seriellen Kommunikation ist ihre Einfachheit in der Verkabelung. Um die Kommunikation zwischen den Geräten herzustellen, wird die Rx-Leitung eines Gerätes mit der Tx-Leitung des anderen Gerätes verbunden und umgekehrt: Die Tx-Leitung des ersten Gerätes wird mit der Rx-Leitung des zweiten Gerätes verbunden. Dadurch entsteht eine Master-Slave-Beziehung zwischen den beiden Geräten, auch wenn die Geräte gleichberechtigt kommunizieren können. Zudem ist UART in den meisten Mikrocontrollern integriert, was die Anwendung vereinfacht und die Kosten minimiert.

Eine detaillierte Beschreibung der UART-Schnittstelle in ESP-Modulen erfolgt in Kapitel 8.6 des Buches von Brandes [4, S. 500].

3.2.4 Inter-Integrated Circuit

Der Inter-Integrated Circuit [12] ist ein serielles, synchrones Kommunikationsprotokoll zur Verbindung von Mikrocontrollern.

I²C-Bus verwendet das Controller-Target-Prinzip (früher Master-Slave-Prinzip). Ein Controller ist das Gerät, das die Datenübertragung auf dem Bus initiiert und die Taktsignale erzeugt. Zu diesem Zeitpunkt wird jedes angesprochene Gerät als Target betrachtet. Es können mehrere Target- und Controller-Geräte am Bus angeschlossen sein, wobei jedes Gerät eine eindeutige Adresse besitzt.

Der Vorteil von I²C liegt in der einfachen Verkabelung, da nur zwei Leitungen für die Kommunikation mehrerer Geräte ausreichen: SDA (Serial Data Line) für die Datenübertragung und SCL (Serial Clock Line) zur Synchronisation.

SDA und SCL sind bidirektionale Leitungen. Wenn der Bus frei ist, sind beide Leitungen auf HIGH gesetzt. Alle Transaktionen beginnen mit einer Startbedingung und enden mit einer Stopbedingung. Die Bedingungen werden immer von dem Controller erzeugt.

Die Kommunikation beginnt mit einer Startbedingung, bei der der Controller die SDA-Leitung auf LOW setzt, während SCL noch HIGH ist. Danach sendet der Controller die Adresse des Targets und ein Lese- oder Schreibbit. Das adressierte Target bestätigt den Empfang mit einem ACK-Bit (Acknowledge), woraufhin der Datenaustausch beginnt. Die Übertragung endet mit einer Stopbedingung, wenn SDA von LOW auf HIGH wechselt, während SCL HIGH bleibt.

⁴Universal Asynchronous Receiver Transmitter

I²C wird häufig in Sensoren, IoT-Geräten und Display-Controllern eingesetzt. Insbesondere in Embedded-Systemen und IoT-Anwendungen ist das Protokoll aufgrund seiner geringen Hardwareanforderungen und flexiblen Adressierung weit verbreitet.

In diesem Projekt wird I²C verwendet, um die Sensoren mit den ESP-Modulen zu verbinden.

3.3 Software

3.3.1 C-Programme

Die Programmiersprache C ist eine prozedurale und leistungsfähige Sprache. Sie wurde im Jahr 1972 von Dennis Ritchie entwickelt. Sie zeichnet sich durch ihre Effizienz und direkte Speicherverwaltung aus. Aufgrund ihrer Eigenschaften wird C vorrangig in der Entwicklung von Betriebssystemen sowie in der Embedded-Programmierung eingesetzt. Insbesondere bei Mikrocontrollern findet C häufig Anwendung.

In diesem Projekt wurde C zur Programmierung der Mikrocontroller verwendet. Für die Entwicklung, Übersetzung und Übertragung des Codes auf die Mikrocontroller wurde die Entwicklungsumgebung **PlatformIO** verwendet.

PlatformIO ist eine Open-Source-Entwicklungsumgebung für eingebettete Systeme, die als Erweiterung in Visual Studio Code verfügbar ist. Sie unterstützt zahlreiche Mikrocontroller-Plattformen, darunter Arduino und ESP-basierte Module.

PlatformIO übernimmt den gesamten Build-Prozess: Der Quellcode wird übersetzt, kompiliert und auf das Gerät übertragen. Über die Konfigurationsdatei „**platformio.ini**“ wird das Projekt konfiguriert. In dieser Datei werden das verwendete Plattform-Framework, das Ziel-Board und die benötigten Bibliotheken angegeben. Weitere erforderliche Build- und Upload-Optionen können dort ebenfalls festgelegt werden.

Durch die Verwendung von **PlatformIO** wird die Entwicklung der Software für die Mikrocontroller effizienter. Der Quellcode kann direkt auf dem Rechner kompiliert und gebildet werden. Somit werden die Fehlübersetzungen vor dem Laden auf das Gerät erkannt.

Durch die Verwendung von **PlatformIO** gestaltet sich die Softwareentwicklung für die Mikrocontroller effizienter. Der Quellcode wird direkt auf dem Rechner kompiliert und überprüft. Somit können die Fehler bereits vor dem Hochladen auf das Gerät erkannt werden.

3.3.2 Python-Programme

Python ist eine weit verbreitete höhere Programmiersprache. Sie wurde 1991 von Guido van Rossum veröffentlicht. Die Sprache wird in verschiedenen Bereichen eingesetzt: von der Webentwicklung und Automatisierung bis hin zu künstlicher Intelligenz und Datenanalyse.

Python ist plattformunabhängig. Der Code von Python-Skripten wird zur Laufzeit in Bytecode übersetzt, der dann von der Python Virtual Machine (PVM) interpretiert und ausgeführt wird. Die PVM übernimmt betriebssystemspezifische Aufgaben wie die Speicherverwaltung, den Zugriff auf Prozessoren und die Nutzung von Systembibliotheken. Dadurch können Python-Programme auf verschiedenen Plattformen ausgeführt werden, sofern eine geeignete PVM für die jeweilige Plattform zur Verfügung steht.

Python zeichnet sich durch eine klare und leicht lesbare Syntax aus. Die Strukturierung von Codeblöcken erfolgt nicht durch geschweifte Klammern, sondern durch Einrückungen.

Python unterstützt die objektorientierte Programmierung (OOP) mit Klassen und Vererbung. Gleichzeitig erlaubt die Sprache weitere Programmierparadigmen wie die prozedurale und funktionale Programmierung. Dadurch eignet sich Python für eine Vielzahl von Softwarearchitekturen und erlaubt es Entwicklern, verschiedene Ansätze flexibel zu kombinieren.

Darüber hinaus bietet Python eine dynamische Typisierung. Das bedeutet, dass der Datentyp einer Variablen erst zur Laufzeit vom Interpreter bestimmt wird. Dies erleichtert die

Entwicklung, kann aber auch zu schwer nachvollziehbaren Fehlern führen.

Ein großer Vorteil von Python ist die große Anzahl an Bibliotheken. Neben der Standardbibliothek gibt es eine große Auswahl an Bibliotheken von Drittanbietern, die speziell für verschiedene Anwendungsbereiche entwickelt wurden. Im Bereich der Webentwicklung steht mit **FastAPI** ein leistungsfähiges Framework zur Erstellung moderner, asynchroner Webanwendungen zur Verfügung, während **SQLAlchemy** eine flexible ORM- und Datenbankbibliothek für effiziente Datenbankzugriffe bietet. Eine aktive Community sorgt für die kontinuierliche Weiterentwicklung und umfassende Dokumentation dieser Werkzeuge.

Python unterstützt die asynchrone Programmierung. Dadurch können langlaufende Prozesse wie Netzwerkanfragen oder Datenbankabfragen ausgeführt werden, ohne den Hauptprozess zu blockieren. Dies wird durch die **asyncio**-Bibliothek und die Verwendung der Schlüsselwörter **async** und **await** ermöglicht.

Eine Funktion wird als asynchron gekennzeichnet, indem sie mit dem Schlüsselwort **async** deklariert wird. Innerhalb einer asynchronen Funktion wird das Schlüsselwort **await** verwendet, um auf die Durchführung einer anderen asynchronen Operation zu warten, ohne den Programmlauf zu unterbrechen. Alle asynchronen Funktionen müssen mit dem Schlüsselwort **await** aufgerufen werden.

Ein Beispiel für eine asynchrone Datenbankabfrage ist im Listing 1 dargestellt. Die Funktion `get_username()` ist eine asynchrone Funktion, die Benutzernamen für die übergebene Benutzer-ID zurückgibt. Die Datenbankabfrage wird mit **await** ausgeführt, damit das Programm während der Wartezeit nicht blockiert wird. Erst wenn das Ergebnis vorliegt, wird der Benutzername zurückgegeben.

Diese Funktion wird innerhalb einer anderen asynchronen Funktion `main()` aufgerufen. Asynchrone Funktionen müssen in einem asynchronen Kontext aufgerufen werden. Deshalb wird die Funktion `main()` mit `asyncio.run()` gestartet. Dadurch wird das asynchrone Programm korrekt ausgeführt.

```
1  import asyncio
2  import datenbank
3
4  async def get_username(user_id):
5      data = await datenbank.get_user(user_id)
6      return data.username
7
8  async def main():
9      username = await get_username(123)
10     print(username)
11
12     asyncio.run(main())
```

Listing 1: Beispiel für asynchrone Abfrage

Diese Methode ist besonders nützlich für Anwendungen, die viele gleichzeitige Anfragen verarbeiten müssen. Durch die nicht-blockierenden Abfragen bleibt die Anwendung responsiv und effizient.

In diesem Projekt werden Python-Skripte sowohl für die Verarbeitung von MQTT-Nachrichten als auch für den Betrieb des Webserver und der Datenbank verwendet.

FastAPI

FastAPI [24] ist ein modernes Webframework für Python, das speziell für die Entwicklung leistungsstarker, asynchroner Webanwendungen und APIs konzipiert wurde. Es basiert auf Starlette für das Web-Handling und Pydantic für die Datenvalidierung.

Ein wesentlicher Vorteil von **FastAPI** ist die eingebaute Unterstützung für asynchronen Code. Dies ermöglicht eine höhere Effizienz, da Anfragen nicht blockierend verarbeitet werden.

Besonders in Anwendungen mit vielen gleichzeitigen Verbindungen und Echtzeitanforderungen ist die asynchrone Programmierung von Vorteil.

Der Aufbau einer Webanwendung mit **FastAPI** folgt einer klar strukturierten Architektur. Zunächst wird die FastAPI-Anwendung mit dem Befehl **FastAPI()** initialisiert. Diese Instanz stellt das zentrale Steuerelement der Anwendung dar. Anschließend werden Router erstellt, um verschiedene API-Endpunkte zu definieren. Ein Router fasst verwandte Endpunkte zusammen und wird dann der Anwendung hinzugefügt.

FastAPI generiert automatisch die API-Dokumentation. Diese wird auf Basis der definierten API-Endpunkte und der Pydantic-Modelle erzeugt. Dadurch erhalten Entwickler eine vollständige, interaktive Dokumentation, die sofort genutzt werden kann, um die API zu testen und zu integrieren.

Die Pydantic-Modelle dienen als Request- und Response-Modelle. Diese Modelle stellen sicher, dass die an die API übergebenen Daten den richtigen Typen und Strukturen entsprechen. Die Validierung und Serialisierung der Daten wird automatisch durchgeführt.

In diesem Projekt wird die Webanwendung mit **FastAPI** entwickelt.

SQLAlchemy

SQLAlchemy [23] ist eine SQL-Bibliothek für Python, die eine flexible und effiziente Schnittstelle zu relationalen Datenbanken bietet.

SQLAlchemy ermöglicht den asynchronen Zugriff auf die Datenbank, wodurch mehrere Abfragen gleichzeitig ausgeführt werden können, ohne den Hauptthread zu blockieren. Für die asynchrone Funktionalität wird die **AsyncEngine** benötigt, die eine **AsyncSession** erstellt. Eine **AsyncSession** wird verwendet, um asynchrone Datenbankabfragen durchzuführen. Änderungen an Datenbankobjekten werden gesammelt und mit dem Befehl **commit()** gespeichert.

SQLAlchemy unterstützt Object-Relational Mapping (ORM). ORM ist eine Programmier-technik, die relationale Datenbanken mit objektorientierter Programmierung (OOP) verbindet. Dabei werden Tabellen als Klassen und Datensätze als Instanzen dieser Klassen dargestellt. Dadurch können Entwickler Datenbankoperationen über Python-Objekte ausführen, ohne direkte SQL-Abfragen schreiben zu müssen.

Der Aufbau einer Webanwendung mit **SQLAlchemy** erfolgt in mehreren Schritten. Zunächst wird eine asynchrone Datenbank-Engine (**AsyncEngine**) erstellt, die die Verbindung zur Datenbank verwaltet. Anschließend wird eine asynchrone Session (**AsyncSession**) konfiguriert, die als Schnittstelle zwischen den Python-Objekten und der Datenbank dient. Die Datenmodelle werden als Python-Klassen definiert, die von **Base** erben und mit Tabellen-Metadaten versehen werden.

In diesem Projekt werden asynchrone Funktionen für den Datenbankzugriff verwendet. Durch den Einsatz von **SQLAlchemy** mit **AsyncSession** können Abfragen parallel ausgeführt werden, ohne den Hauptprozess zu blockieren.

3.3.3 Webentwicklung

Die Entwicklung moderner Webanwendungen basiert auf drei zentralen Technologien: HTML, CSS und JavaScript. Diese bilden die Grundlage für die Erstellung interaktiver und ansprechender Webseiten. HTML dient der strukturellen Definition der Inhalte, CSS sorgt für die visuelle Gestaltung und JavaScript ermöglicht dynamische Funktionen und Interaktionen. Durch das Zusammenspiel dieser Technologien können komplexe, dynamische Webanwendungen realisiert werden.

HTML

Die Hypertext Markup Language (HTML) ist die standardisierte Beschreibungssprache zur Strukturierung und Darstellung von Webseiteninhalten. Sie bildet die Grundlage jeder Webseite und definiert deren logischen Aufbau durch eine hierarchische Struktur von Elementen.

HTML ermöglicht die Strukturierung von Webseiten durch die Definition von Elementen und deren Beziehungen. Diese Elemente werden von Webbrowsern interpretiert und in eine visuelle Darstellung umgesetzt. Darüber hinaus unterstützt HTML die Einbindung externer Ressourcen wie CSS zur Gestaltung und JavaScript zur Implementierung dynamischer Funktionen.

HTML-Dokumente folgen einer baumartigen Struktur und bestehen aus ineinander verschachtelten Elementen, die durch sogenannte Tags gekennzeichnet sind. Jedes HTML-Dokument beginnt mit einer `<!DOCTYPE>`-Deklaration, die den Dokumenttyp definiert.

Die Grundstruktur eines HTML-Dokuments besteht aus folgenden Hauptelementen:

- `<html>`: Das Wurzelement. Enthält den gesamten HTML-Code.
- `<head>`: Der Kopf des Dokuments. Enthält die Metadaten, wie Seitentitel, Zeilenkodierung, Pfade zu den Stylesheets und Skripts.
- `<body>`: Der Hauptinhalt der Seite. Enthält alle Elemente, die an der Webseite angezeigt werden.

Neben diesen Basiselementen enthält HTML eine Vielzahl weiterer Elemente zur Strukturierung von Webseiten. Dazu gehören das `<div>`-Element als Container und das `<button>`-Element für interaktive Schaltflächen.

CSS

Die Cascading Style Sheets (CSS) sind eine Stylesheet-Sprache zur Gestaltung von HTML-Dokumenten. Sie ermöglichen die Trennung von Inhalt und Design, wodurch Webseiten flexibler und effizienter angepasst werden können.

CSS definiert das visuelle Erscheinungsbild von HTML-Elementen durch Eigenschaften wie Farben, Schriften, Abstände und Layouts. Es folgt einer hierarchischen Struktur, in der Regeln kaskadierend angewendet werden. Die CSS-Regeln werden hierarchisch angewendet, wobei übergeordnete Elemente ihre Stile an untergeordnete weitergeben. Falls mehrere Regeln auf dasselbe Element zutreffen, bestimmt die Spezifität der Selektoren, welche Regel Vorrang hat.

CSS kann auf drei Arten eingebunden werden: direkt im `style`-Attribut eines HTML-Elements, innerhalb eines `<style>`-Elements im `<head>`-Bereich oder am häufigsten in einer separaten `.css`-Datei gespeichert und über das `<link>`-Element in das HTML-Dokument eingebunden.

Neben der grundlegenden Formatierung unterstützt CSS Positionierungstechniken, die eine präzise Anordnung von Elementen ermöglichen. Darüber hinaus ermöglicht CSS die Implementierung von Animationen und responsiven Designs, so dass sich Webseiten an unterschiedliche Bildschirmgrößen und Geräte anpassen können.

Eine Alternative zu klassischem CSS ist **Tailwind CSS**. Es vereinfacht die Gestaltung von Webseiten durch vordefinierte Klassen.

Im **Tailwind CSS** hat jede Klasse eine spezifische Funktion, so dass komplexe Designs ohne eigene Stylesheets möglich sind. Außerdem bietet es eine Konfigurationsdatei, in der Farben, Abstände und andere Designparameter individuell angepasst werden können.

Anstatt für jedes Element individuelle CSS-Regeln zu schreiben, werden Style-Sheets direkt durch CSS-Klassen im `class`-Attribut eines HTML-Elements definiert. Beispielsweise erzeugt die Klasse `class="bg-green-500 text-blue p-4"` ein grünes Hintergrundfeld mit blauer Schrift und einem Innenabstand von 16 Pixeln.

Ein Vorteil von Tailwind CSS ist die hohe Flexibilität und Wiederverwendbarkeit der Klassen. Das Styling kann direkt in den Komponenten definiert werden. Außerdem bietet Tailwind

CSS eine **Purge-Funktion**, die nicht verwendete Klassen aus der endgültigen CSS-Datei entfernt, um die Ladezeiten zu optimieren.

JavaScript

JavaScript ist eine dynamische Programmiersprache, die zur Interaktivität und Behandlung von Webseiteninhalten verwendet wird. Sie ermöglicht die Erstellung dynamischer, benutzerinteraktiver Webseiten. Mit JavaScript können HTML-Elemente zur Laufzeit verändert und Reaktionen auf Benutzereingaben definiert werden. Die Grundstruktur von JavaScript basiert auf Anweisungen, die sequentiell ausgeführt werden.

JavaScript ist eine ereignisgesteuerte Sprache, in der viele Prozesse durch Ereignisse wie Klicks, Tastatureingaben oder Zeitabläufe ausgelöst werden. In JavaScript wird der Code in Funktionen organisiert, die bestimmte Aufgaben ausführen. Diese Funktionen können bei Bedarf aufgerufen werden.

JavaScript unterstützt asynchronen Programmcode. Asynchrone Funktionen ermöglichen lange Operationen im Hintergrund, ohne die Benutzeroberfläche zu blockieren. Asynchrone Funktionen werden mit dem Schlüsselwort **async** deklariert. Das Schlüsselwort **await** wird verwendet, um auf das Ergebnis einer anderen asynchronen Funktion zu warten. Ein einfaches Beispiel für eine asynchrone Funktion ist:

```
async function ladeDaten() {  
  let response = await fetch("/get_data");  
  let data = await antwort.json();  
  console.log(daten);  
}
```

Das Schlüsselwort **await** sorgt dafür, dass der JavaScript-Interpreter wartet, bis die Daten abgerufen und in der Variablen **data** gespeichert wurden, bevor der Code fortgesetzt wird. Dadurch wird die Blockierung des Codes vermieden. Gleichzeitig wird es sichergestellt, dass der gesamte Prozess abgeschlossen ist, bevor mit den Daten weitergearbeitet wird.

Eine zentrale Funktion von JavaScript ist die Behandlung des Document Object Model (DOM). Das DOM stellt die Webseite als Baumstruktur dar, wobei jedes HTML-Element als Knoten in diesem Baum betrachtet wird. Mit JavaScript können diese Knoten verändert, hinzugefügt oder entfernt werden, um die Darstellung der Seite zu beeinflussen.

WebSocket

WebSocket ist ein Kommunikationsprotokoll, das eine dauerhafte, bidirektionale Verbindung zwischen einem Client und einem Server ermöglicht. Im Gegensatz zu klassischen HTTP-Verbindungen, die nach jeder Anfrage und Antwort geschlossen werden, bleibt bei WebSocket die Verbindung offen, so dass beide Parteien jederzeit Daten austauschen können. Dies ermöglicht eine Echtzeit-Kommunikation, ohne dass ständig neue Verbindungen aufgebaut werden müssen.

WebSocket ermöglicht das Senden von Daten in beide Richtungen: vom Client zum Server und umgekehrt. Dies ist besonders nützlich für Anwendungen, die eine kontinuierliche Datenübertragung benötigen, wie z.B. Echtzeit-Datenvisualisierungen.

In JavaScript kann WebSocket einfach mit der WebSocket-API verwendet werden. Eine WebSocket-Verbindung wird hergestellt, indem ein neues **WebSocket**-Objekt instanziiert wird, das die URL des WebSocket-Servers als Parameter erhält. Sobald die Verbindung hergestellt ist, können Nachrichten gesendet und empfangen werden.

4 Einrichtung der Hardwarekomponenten

In diesem Kapitel wird die Einrichtung der im System verwendeten Hardwarekomponenten beschrieben.

4.1 Raspberry Pi

Der Raspberry Pi dient als zentraler Server des Systems. Als Netzwerkzentrale gewährleistet er eine stabile und sichere Kommunikation zwischen den Komponenten, auch in Umgebungen ohne Internetzugang. Er stellt einen WLAN-Zugriffspunkt zur Verfügung, über den sich die Geräte mit dem Server verbinden.

Die Kommunikation zwischen den Geräten erfolgt über das MQTT-Protokoll. Der Raspberry Pi hostet den MQTT-Broker auf Basis von **Mosquitto**. Die Geräte senden ihre Daten in individuelle Topics. Diese Daten werden von einem asynchronen MQTT-Client empfangen und verarbeitet.

Der genaue Ablauf für die Einrichtung des Raspberry Pi ist in dem Bericht zu der Projektarbeit 2 [2], Kapitel 2, beschrieben.

In diesem Projekt wird der Raspberry Pi ausschließlich über ein Netzkabel mit dem Internet verbunden, um die benötigten Linux-Pakete und Python-Bibliotheken herunterzuladen. Nach der Einrichtung ist das System nur über ein lokales, vom Internet getrenntes Netzwerk erreichbar. Dies erhöht die Sicherheit des Systems, da Angriffe aus dem globalen Netzwerk somit ausgeschlossen sind.

Der Raspberry Pi hostet zudem den API-Webserver, der als Schnittstelle für die Interaktion mit dem System dient. Die API ist mit **FastAPI** implementiert und läuft auf dem ASGI-Server **Uvicorn**. **Uvicorn** ermöglicht eine effiziente, asynchrone Verarbeitung von Anfragen und sorgt für eine hohe Leistungsfähigkeit.

Die von den Geräten empfangenen Daten werden durch den asynchronen MQTT-Client verarbeitet und über **WebSocket** direkt an die API weitergeleitet. Dadurch wird eine Echtzeitübertragung an die Benutzeroberfläche ermöglicht, ohne dass die Daten zwischengespeichert oder erneut abgefragt werden müssen.

Diese Architektur ermöglicht die effiziente und schnelle Interaktion mit dem System. Das System ist zwar nur im lokalen Netzwerk verfügbar. Da keine Cloud-Dienste erforderlich sind, bietet das System eine erhöhte Sicherheit und ist von der Internetverbindung unabhängig.

4.2 Arduino

Der Arduino mit den angeschlossenen Sensoren stellt die Sicherheitskomponente des Systems dar.

Die am Arduino angeschlossenen Sensoren erkennen die Gefahren, wie Gas oder Feuer. Im Falle einer Gefahrensituation sendet der Arduino eine Warnmeldung an den Server und aktiviert den Alarm.

Zusätzlich dient der Arduino als Zugangskontrolle zum Haus. Über ein angeschlossenes Tastenfeld kann eine PIN eingegeben werden, um den Zugang zum Haus zu sichern. Die genaue Anbindung der Sensoren und Aktoren ist in der Projektdokumentation PA1 [1] beschrieben.

Da der Arduino kein WLAN-Modul besitzt, wird für die drahtlose Kommunikation ein ESP8266-Modul verwendet. Das Modul wird über eine UART-Schnittstelle mit dem Arduino verbunden. Der Arduino sendet die Daten für die MQTT-Nachrichten über **Serial** an das ESP8266. Die Daten werden im Format „**topic:message**“ gesendet. Das Modul verarbeitet diese und sendet die generierte MQTT-Nachricht.

Die detaillierte Beschreibung der Verbindung von Arduino und ESP8266 ist in dem Bericht zu meiner Projektarbeit 2 [2] Kapitel 4.1 zu finden.

In diesem Projekt wird der Programmcode des Arduinos erweitert, um eine optimierte Interaktion mit der API zu ermöglichen.

Der aktuelle Status der an dem Arduino angeschlossenen Sensoren wird einmal pro Sekunde an dem Server gesendet. Der Status wird in einem JSON-Objekt strukturiert, der für das Senden über `Serial` in einem String serialisiert wird. Das JSON-Objekt hat folgende Struktur:

```
{
  "fire": "OK",
  "gas": "OK",
  "lock": "Closed",
  "pin": pin_status
}
```

Anschließend werden das Haupttopic „data“ und das serialisierte JSON-Objekt an das ESP8266 gesendet.

Wird eine Gefahr erkannt, löst der Arduino sofort den Alarm aus. Weiterhin sendet der Arduino das Topic „alarm“ und die entsprechende Meldung an das ESP8266.

Die PIN-Eingabe wird über MQTT übertragen. Bei der Eingabe der PIN wird eine MQTT-Nachricht an den Server gesendet. Dabei wird übermittelt, ob die PIN richtig oder falsch eingegeben wurde. Wurde die PIN geändert, wird dies ebenfalls an den Server gesendet.

Außerdem muss der Arduino auf die eingehende Befehle reagieren. Die eingehenden Daten werden mit der Funktion „`readSerialData()`“ ausgelesen und verarbeitet.

Beim Empfang des Befehls „handshake“ vom ESP8266, wird ein JSON-Objekt mit strukturierten Konfigurationsdaten erstellt und zusammen mit dem Topic „handshake“ an dem ESP8266 gesendet.

Die Konfigurationsdaten haben folgende Struktur:

```
{
  "data_fields": ["fire", "gas", "pin"],
  "commands": ["alarm_off"]
}
```

Beim Empfang des Befehls „alarm_off“ wird der Alarmton ausgeschaltet.

Der Arduino übernimmt die sicherheitskritischen Aufgaben des Systems. Die Meldungen über gefährliche Vorgänge werden unverzüglich an den Server gesendet.

4.3 Einrichtung der ESP-Module

In diesem Projekt werden drei verschiedene WLAN-fähige ESP-Module verwendet: ESP8266, ESP32 und der auf dem ESP32 basierende M5Stick. Jedes Modul erfüllt spezifische Aufgaben, aber alle Module müssen sowohl mit dem WLAN als auch mit dem MQTT-Broker verbunden sein. Daher ist die Implementierung der `setup()`-Funktion für alle drei Module ähnlich aufgebaut.

Die Setup-Funktion wird beim Start des ESP-Moduls aufgerufen. Zunächst wird das Modul mit der Funktion `setup_esp` entsprechend eingerichtet. Je nach Modul umfasst diese Funktion verschiedene Anfahrtroutinen. Dazu gehört unter anderem die Initialisierung der seriellen oder I2C-Kommunikation, die Einrichtung der angeschlossenen Sensoren o. ä.

Verbindung mit WLAN

Anschließend werden mit der Funktion `connect_wifi_mqtt` (Listing 2) die WLAN- und MQTT-Verbindungen aufgebaut. Zunächst wird versucht die WLAN-Daten aus dem EEPROM zu laden. Bevor die Daten aus dem EEPROM gelesen werden können, muss es mit einer Buffer-Größe von 64 bit initialisiert werden (Zeile 3).

Es werden zwei Felder vom Typ `char` mit vordefinierten maximalen Längen zum Speichern der Daten aus dem EEPROM erstellt (Zeile 4-5). Der Name des WLAN-Netzes (SSID) wird ab Adresse 0 bis 31 des EEPROM ausgelesen und mit der Funktion `EEPROM.get(0, eeprom_ssid)` im Char-Feld gespeichert. Das Passwort wird ab Adresse 32 aus dem EEPROM ausgelesen und in das zweite Char-Feld gespeichert.

Anschließend wird mit der Boolean-Funktion `is_valid_string()` geprüft, ob die aus dem EEPROM abgelesenen Daten sinnvoll sind. Die Funktion gibt den Wert `False` zurück, wenn ein Standard-EEPROM-Zeichen mit dem Wert `0xFF` gefunden wurde. Dies bedeutet, dass EEPROM noch leer ist und es keine gültigen WLAN-Daten im EEPROM vorhanden sind.

```
1  void connect_wifi_mqtt()
2  {
3      EEPROM.begin(64);
4      char eeprom_ssid[32] = {0};
5      char eeprom_password[32] = {0};
6      EEPROM.get(0, eeprom_ssid);
7      EEPROM.get(32, eeprom_password);
8      EEPROM.end();
9      // Daten gefunden
10     if (is_valid_string(eeprom_ssid, MAX_SSID_LENGTH) &&
11         is_valid_string(eeprom_password, MAX_PASSWORD_LENGTH))
12     {
13         // verbinden mit WLAN
14         if (wifi_connect(eeprom_ssid, eeprom_password))
15         {
16             mqtt_connect();
17         }
18         else // die WLAN-Verbindung is schiefgelaufen
19         { setup_ap(); }
20     }
21     else // keine WLAN-Daten gefunden
22     { setup_ap(); }
23 }
```

Listing 2: ESP: `connect_wifi_mqtt()`

Sind die gültigen Werte im EEPROM gespeichert, versucht das Modul, sich mit diesen Zugangsdaten mit dem WLAN zu verbinden (Zeile 13). Die Funktion `connect_wifi()` schaltet das WLAN-Modul in den Modus „Station“ um, was die Verbindung mit anderen WLAN-Netzwerken erlaubt. Danach wird es versucht mit dem WLAN mit übergebenen Zugangsdaten zu verbinden. Die Anzahl der Versuche zur Erstellung der Verbindung ist in der Variable `wifi_repeat` gespeichert. Ein Mal pro Sekunde versucht das Modul sich mit dem WLAN-Netzwerk zu verbinden. Wenn die Verbindung erfolgreich erstellt wurde, gibt die Funktion ein `True` zurück. Soll die Anzahl der Versuche überschritten sein, so wird ein `False` zurückgegeben.

Aktualisierung der gespeicherten WLAN-Daten

Sind die aus dem EEPROM ausgelesenen Daten ungültig oder kann keine erfolgreiche

WLAN-Verbindung aufgebaut werden, müssen die WLAN-Daten aktualisiert werden. Zu diesem Zweck wird ein Webserver zur Verfügung gestellt, der ein Formular zur Eingabe der WLAN-Daten bereitstellt.

Die Funktion `setup_ap()` (Listing 4) stellt den Webserver bereit. Das WLAN-Modul wird in den Modus „Access Point“ versetzt. Die Konfiguration des WLAN-Zugangspunktes erfolgt über die Befehle `WiFi.softAP()` und `WiFi.softAPConfig()` (Zeile 3-4), wobei die Parameter des Access Points in Listing 3 definiert sind.

Der Webserver wird durch das Objekt `server` der Klasse `AsyncWebServer` zur Verfügung gestellt und mit dem Befehl `server.begin()` gestartet (Zeile 36).

```
1  const char AP_SSID[] = DEVICE_ID;
2  const char AP_PASSWORD[] = "setupes32";
3  // Gleich fuer alle ESP-Module
4  IPAddress ap_ip(10, 0, 0, 1);
5  IPAddress ap_gateway(10, 0, 0, 1);
6  IPAddress ap_subnet(255, 255, 255, 0);
7  AsyncWebServer server(80);
```

Listing 3: ESP: Access Point

```
1  void setup_ap()
2  {
3      WiFi.mode(WIFI_AP);
4      WiFi.softAP(AP_SSID, AP_PASSWORD);
5      WiFi.softAPConfig(ap_ip, ap_gateway, ap_subnet);
6      server.on("/", HTTP_GET, [](AsyncWebServerRequest *request)
7      { request->send(200, "text/html", html_page); });
8
9      server.on("/save", HTTP_POST, [](AsyncWebServerRequest *request)
10     {
11         // GET Daten aus Input
12         String ssid = request->getParam("ssid", true)->value();
13         String password = request->getParam("password", true)->value();
14         if (ssid.length() > MAX_SSID_LENGTH - 1 ||
15             password.length() > MAX_PASSWORD_LENGTH - 1)
16         {
17             ap_on = false;
18             return;
19         }
20         // String in char[]
21         char new_ssid[MAX_SSID_LENGTH] = {0};
22         strncpy(new_ssid, ssid.c_str(), MAX_SSID_LENGTH - 1);
23         char new_password[MAX_PASSWORD_LENGTH] = {0};
24         strncpy(new_password, password.c_str(), MAX_PASSWORD_LENGTH - 1)
25         ;
26
27         // in EEPROM speichern
28         EEPROM.begin(64);
29         EEPROM.put(0, new_ssid);
30         EEPROM.put(32, new_password);
31         EEPROM.commit();
32         EEPROM.end();
33
34         request->send(200, "text/html", "WiFi saved. Rebooting...");
```

```

34     delay(1000);
35     ESP.restart(); });
36     ap_on = true;
37     server.begin();
38 }

```

Listing 4: ESP: setup_ap

Die HTTP-Routen werden innerhalb der Funktion `server.on()` festgelegt. Beim Aufruf der Root-Route („/“) wird eine HTTP-GET-Anfrage bearbeitet, die mit einer HTML-Seite beantwortet wird (Zeile 6). Diese Seite enthält ein Formular mit Input-Felder zur Eingabe der WLAN-Zugangsdaten. Der Benutzer soll der Name des Netzwerks und das Passwort eingeben.

Wenn der Benutzer auf die Taste „Submit“ drückt, wird eine HTTP-POST-Anfrage an die Route „/save“ gesendet. Die Zugangsdaten werden aus dem `request`-Objekt ausgelesen und auf ihre maximale Länge überprüft. Wenn die Daten gültig sind, werden sie im EEPROM gespeichert. Der EEPROM wird initialisiert, die neuen Zugangsdaten werden gespeichert, und die Änderungen mit `EEPROM.commit()` übernommen (Zeile 26-30). Anschließend wird ein Hinweis zurückgegeben, dass die Daten übernommen wurden. Das ESP-Modul wird mit dem Befehl `ESP.restart()` neu gestartet, um die neuen WLAN-Daten zu verwenden.

Verbindung mit MQTT-Broker

Nach erfolgreicher Verbindung mit dem WLAN, wird die Funktion `mqtt_connect()` (Listing 5) aufgerufen, um die Verbindung zum MQTT-Broker zu erstellen. Zunächst werden die Topics zum Senden und Empfangen der Nachrichten mit der Funktion `set_topics()` erstellt (Zeile 3). Dabei werden die Topics mit **Geräte-ID** für jedes Modul individuell formuliert.

Der `mqttClient` ist ein Objekt der Klasse `PubSubClient`. Die Funktion `setServer()` erhält als Parameter die IP-Adresse des MQTT-Brokers sowie den Port, auf dem der Broker lauscht (Zeile 4). Diese Parameter sind als Konstanten im Programmcode definiert. Die `callback`-Funktion muss definiert werden und wird beim Empfang einer MQTT-Nachricht ausgeführt.

Es wird maximal `wifi_repeat`-mal versucht, die Verbindung zum MQTT-Broker herzustellen. Sollte dies nicht erfolgreich sein, wird `False` zurückgegeben. Andernfalls abonniert der `mqttClient` das `SUBSCRIBE_TOPIC` und die Funktion gibt `True` zurück.

```

1  void mqtt_connect()
2  {
3      set_topics();
4      mqttClient.setServer(WiFi.gatewayIP(), MQTT_PORT);
5      mqttClient.setCallback(callback);
6      // Verbinden mit dem MQTT-Broker
7      for (uint8_t i = 0; i < wifi_repeat; i++){
8          if (mqttClient.connect(DEVICE_ID))
9              {
10                 mqttClient.subscribe(SUBSCRIBE_TOPIC);
11                 return True;
12             }
13             else {delay(1000);}
14         }
15         return False;
16     }f

```

Listing 5: ESP: mqtt_connect

Verwaltung eingehender MQTT-Nachrichten

Die Funktion `callback` wird beim Empfang einer MQTT-Nachricht aufgerufen. In dieser Funktion sind die Reaktionen auf den erhaltenen Nachrichten aus dem Topic „`command/ID`“ definiert. Abhängig vom Gerät wird diese Funktion angepasst und erweitert.

Der Text der empfangenen MQTT-Nachricht wird als `byte payload` an die Funktion übergeben und zunächst im `String message` gespeichert. Dieser enthält die auszuführenden Befehle, die je nach Gerät unterschiedlich interpretiert werden können. Jedes Gerät muss jedoch auf das Kommando „`handshake`“ reagieren.

Wenn eine Handshake-Nachricht empfangen wurde, muss das Gerät mit seinen Konfiguration antworten (Listing 6). Zunächst werden zwei Arrays definiert: eines für die relevanten Datenfelder und eines für die möglichen Befehle. Im Array „`data_fields`“ sind die Schlüssel gelistet, die das Modul in das Topic „`data/ID`“ sendet. Im Array „`commands`“ sind die Befehle gelistet, die das Modul ausführen kann. Je nach Modul variiert der Inhalt. Diese Arrays werden in einem JSON-Dokument strukturiert, wobei die Datenfelder und Befehle jeweils als separate Arrays innerhalb des Dokuments abgelegt werden:

```
{
  "data_fields": [ "temperature", "humidity"],
  "commands": [ "light_on", "light_off"]
}
```

Anschließend wird das JSON-Dokument serialisiert und in eine Zeichenkette umgewandelt. Diese serialisierte Nachricht wird dann über die MQTT-Verbindung an das Topic „`handshake/ID`“ gesendet.

```
1  void callback(char *topic, byte *payload, unsigned int length)
2  {
3      if (message == "handshake")
4      {
5          const char *data_fields[] = {"temperature", "humidity"};
6          const char *commands[] = {"light_on", "light_off"};
7
8          char json[128];
9          JsonDocument data;
10         JsonArray json_df = data.createNestedArray("data_fields");
11         JsonArray json_commands = data.createNestedArray("commands");
12
13         for (const char* df: data_fields){ json_df.add(df); }
14         for (const char* a: commands){ json_actions.add(a); }
15
16         serializeJson(data, json, sizeof(json));
17         mqttClient.publish(HANDSHAKE_TOPIC, json);
18     }
19 }
```

Listing 6: ESP: callback

Diese Setup-Konfiguration wird bei allen ESP-Modulen verwendet. Je nach Modul wird diese Funktion erweitert sein. Beispielsweise wird bei Modulen mit Display angezeigt, ob die Verbindung aufgebaut wurde.

4.3.1 ESP8266

Das ESP8266 stellt die WLAN-Verbindung für den Arduino her und übernimmt die Kommunikation über das MQTT-Protokoll. Er empfängt Steuerbefehle sowohl vom Arduino über die serielle Schnittstelle als auch direkt über MQTT. Empfangene Daten werden über die serielle Schnittstelle an den Arduino weitergeleitet.

Die genaue Umsetzung und Verwendung des Moduls im System ist in meinem Bericht zur Projektarbeit 2 [2] Kapitel 4.1 beschrieben.

In diesem Projekt wird die Funktionalität des Moduls erweitert.

Beim Start des Moduls wird die Funktion `setup()` ausgeführt. Dabei wird versucht, die WLAN- und MQTT-Verbindung zu erstellen. Wenn die Verbindung nicht aufgebaut werden kann, stellt das Modul ein Webserver zur Verfügung. Über den Webserver können die aktuelle WLAN-Daten des Zugangspunktes an Modul übergeben werden.

Die Kommunikation zwischen ESP8266 und Arduino erfolgt über eine UART-Schnittstelle. Der Arduino sendet seine Daten für die Übertragung über die MQTT-Verbindung. Die Daten werden im Format „`topic:message`“ gesendet.

Bei jedem Programmdurchlauf wird die Funktion `readSerialData()` auf dem ESP8266 aufgerufen. Diese Funktion liest die Daten aus der serielle Schnittstelle aus. Nach Empfang der Daten, wird das Topic mit der Geräte-ID erstellt und die MQTT-Nachricht wird an dem MQTT-Broker gesendet.

Die Verarbeitung der empfangenen MQTT-Nachrichten erfolgt in der Funktion `callback()`. Das ESP8266 abonniert nur das Topic „`command/arduino`“. Beim Empfang einer MQTT-Nachricht, wird der Text in ein `String` konvertiert und über `Serial` an Arduino weitergeleitet.

4.3.2 M5Stick

Das M5Stick wird zur Überwachung der Licht- und Luftbedingungen eingesetzt. Dem Modul wird die Geräte-ID „`m5stick`“ zugewiesen.

Das Modul verfügt über einen integrierten Akku und kann beispielsweise außen am Fenster platziert werden. Dadurch kann das System die Wetterbedingungen erfassen und entsprechend darauf reagieren.

Am Modul sind die Sensoren BME680 und VCNL4040 über I²C angebunden. Der BME680 erfasst Umweltdaten: Temperatur, Luftfeuchtigkeit und Gaswiderstand. Der VCNL4040 dient als Lichtsensor und misst Lichtbedingungen: Umgebungshelligkeit (Ambient Light), weißes Licht sowie Annäherung (Proximity). Diese Daten werden an dem Server via MQTT-Nachrichten gesendet.

Für die Verwendung der beiden Sensoren im Programmcode müssen die entsprechenden Bibliotheken importiert werden und die Klassenobjekten für die Sensoren erstellt werden (Listing 7).

```
1  #include <Adafruit_BME680.h>
2  #include <Adafruit_VCNL4040.h>
3  Adafruit_BME680 bme_sensor;
4  Adafruit_VCNL4040 vcnl4040 = Adafruit_VCNL4040();
```

Listing 7: M5Stick: Sensoren

Beim Start des Moduls wird die Funktion `setup()` aufgerufen. Das Modul wird mit dem Befehl `M5.begin()` initialisiert. Zusätzlich zu den im letzten Abschnitt beschriebenen Einrichtungen müssen auch die beiden angeschlossenen Sensoren eingerichtet und in Betrieb genommen werden. Zum Einstellen der Sensoren werden im Setup die Funktionen `vcnl_setup()` und `bme_setup()` aufgerufen. Diese Funktionen stellen die I²C-Verbindung zu den Sensoren her und initialisieren diese.

Anschließend wird die WLAN- und MQTT-Verbindung aufgebaut. Der Status der Verbindung wird mit der Funktion `show_connections()` bei jedem Programmdurchlauf auf dem Display des Moduls angezeigt. Sollte die Verbindung abbrechen oder nicht aufgebaut werden, wird die entsprechende Meldung ebenfalls angezeigt.

Bei jedem Programmdurchlauf zeigt die Funktion `show_sensor_data()` die von den Sensoren erfassten Daten auf dem Display an.

Mit der Funktion `send_mqtt_data()` werden die Sensordaten erfasst und an dem MQTT-Broker gesendet. Zunächst werden die Messwerte der Sensoren ausgelesen und in einem JSON-Objekt zusammengefasst. Der BME680 liefert die Temperatur in °C sowie die Luftfeuchtigkeit in %. Der VCNL4040 erfasst die Umgebungshelligkeit in Lux abgelesen. Anschließend wird das JSON-Objekt serialisiert und im Topic „`data/m5stick`“ an den MQTT-Broker gesendet.

Das auf dem Modul laufenden MQTT-Client abonniert das Topic „`command/m5stick`“. Beim Empfang einer MQTT-Nachricht aus diesem Topic mit dem Text „`handshake`“, werden die Konfigurationsdaten in das Topic „`handshake/m5stick`“ gesendet. Die Konfigurationsdaten des Moduls sind wie folgt strukturiert:

```
{
  "data_fields": ["temperature", "humidity", "light"],
  "commands": ["none"]
}
```

4.3.3 ESP32

Das ESP32 wird zur Überwachung von Luftbedingungen im Raum sowie zur Steuerung der Beleuchtung verwendet. Das Modul besitzt die Geräte-ID „`esp32`“.

Ein Display [10] und ein BME680-Sensor sind über die I²C-Schnittstelle an das ESP32 angeschlossen. Auf dem Display werden Temperatur, Luftfeuchtigkeit und der Verbindungsstatus angezeigt. Zur Ansteuerung des Displays wird die Bibliothek `MycilaEasyDisplay` [20] verwendet.

Zusätzlich ist eine Diode an dem GPIO-Pin 4 des ESP32 angeschlossen, die die Raumbeleuchtung simuliert.

Beim Start des Moduls wird die Funktion `setup()` aufgerufen. In dieser Funktion wird zunächst das Display mit den Befehlen `display.begin()` und `display.setActive(true)` initialisiert und aktiviert. Der BME680-Sensor wird in der Funktion `bme_setup()` initialisiert. Anschließend werden die WLAN- und MQTT-Verbindungen standardmäßig aufgebaut.

Nach dem Setup werden bei jedem Programmdurchlauf die Funktionen `show_connections()` und `show_sensordata()` aufgerufen.

Die Funktion `show_connections()` gibt den aktuellen Status der WLAN- und MQTT-Verbindung auf dem Display aus. Bei einem Verbindungsausfall wird eine Meldung mit den Daten des erstellten Zugangspunktes angezeigt.

Die Funktion `show_sensordata()` liest die aktuellen Werte für die Luftfeuchtigkeit und die Raumtemperatur vom BME680-Sensor aus und stellt diese auf dem Display dar.

Die Funktion `send_mqtt_sensor_data()` liest die von dem BME680-Sensor erfassten Daten aus und überträgt diese über die bestehende MQTT-Verbindung. Die Sensordaten werden zunächst ausgelesen und in den lokalen Variablen gespeichert. Anschließend wird ein JSON-Objekt erzeugt, in dem die Messwerte den entsprechenden Feldern zugewiesen werden. Dieses JSON-Objekt wird serialisiert und an das Topic „`data/esp32`“ gesendet.

Der auf dem Modul laufende MQTT-Client abonniert das Topic „`command/esp32`“. Beim Empfang einer MQTT-Nachricht aus diesem Topic mit dem Text „`handshake`“, antwortet der

Client mit den Konfigurationsdaten. Die Konfigurationsdaten des Moduls haben folgende Struktur:

```
{
  "data_fields": ["temperature", "humidity"],
  "commands": ["light_on", "light_off"]
}
```

Aus den Konfigurationsdaten ergibt sich, dass das Modul die Werte für die Temperatur und die Luftfeuchtigkeit liefert sowie die Befehle zum Ein- und Ausschalten des Lichts ausführen kann. Beim Empfang einer Nachricht mit dem Text „light_on“ wird der GPIO-Pin 4 auf HIGH gesetzt, wodurch die angeschlossene Diode eingeschaltet wird. Beim Empfang einer Nachricht mit dem Text „light_off“ wird der GPIO-Pin 4 auf LOW gesetzt und die Diode entsprechend ausgeschaltet.

Das ESP32 liefert kontinuierlich die erfassten Umweltdaten und reagiert auf die eingehenden Steuerbefehle. Dadurch ist das Modul flexibel in verschiedenen Szenarien einsetzbar.

5 Softwareentwicklung

In diesem Abschnitt wird die Aufbau und Entwicklung der Software für das System beschrieben. Für die Entwicklung wurden Programmiersprachen Python und JavaScript mit mehreren Bibliotheken sowie die Markierungssprache HTML und CSS für die Visualisierung der Webseite verwendet.

Das Programm läuft auf dem Raspberry Pi. Der Programmcode ist in dem Projektordner unter `bachelor/code/spa` zu finden. Das Programm ist modular aufgebaut, wobei jedes Paket (Package) in einem eigenen Unterordner organisiert ist und eine spezifische Aufgabe erfüllt. Die Struktur des Programms sieht wie folgt aus:

```
bachelor/code/spa/  
|  
|--- app/  
|   |--- config/  
|   |--- db/  
|   |--- mqtt/  
|   |--- webapp/  
|   |--- __init__.py  
|   |--- __main__.py  
|  
|--- requirements.txt  
|--- tailwind.config.js
```

- **app/** — Hauptmodul des Programms. Enthält die Startdatei `__main__.py` sowie eine Initialisierungsdatei `__init__.py`, um das gesamte Modul als Package erkennbar zu machen.
- **config/** — Enthält die Einstellungen für die Datenbankverbindungen, MQTT-Client, Logger und den Webserver.
- **db/** — Verwaltet der Datenbank. Enthält die Funktionen für die Verbindung zur Datenbank, die Erstellung von Tabellen und die Interaktion mit der Datenbank.
- **mqtt/** — Enthält die Logik für die Kommunikation über das MQTT-Protokoll. Enthält Code für den Empfang, die Verarbeitung und das Senden von MQTT-Nachrichten.
- **webserver/** — Stellt die API-Endpunkte und eine Webseite zur Verfügung. Enthält Klassen zur Verwaltung der Benutzerinteraktionen und zur Bereitstellung der Webseite.

Zuerst müssen die benötigten Module auf dem Server installiert werden. Die Module sind in der Datei `requirements.txt` aufgelistet. Mit dem Befehl werden die Module heruntergeladen und installiert:

```
pip install -r requirements.txt
```

5.1 Datenbank

Struktur der Datenbank

Die Datenbank für dieses Projekt wurde mit `SQLAlchemy` implementiert. Ziel war es, eine relationale Datenbank zu erstellen, die modular und flexibel aufgebaut ist, sowie die Möglichkeit bietet die Datenbankoperationen ohne direkten SQL-Abfragen durchzuführen.

Die Funktionen zur Verwaltung der Datenbank sind in dem Paket „db“ zusammengefasst. Dieses Paket bündelt alle relevanten Komponenten, wie die Konfiguration der Datenbankverbindung, das Session-Management sowie die Definition der Datenbankmodelle. Durch diese modulare Struktur kann das Datenbankpaket in anderen Klassen importiert werden, um eine zentrale und einheitliche Interaktion mit der Datenbank zu ermöglichen.

Die Struktur des Packets sieht wie folgt aus:

```

bachelor/code/spa/db/
|
|--- __init__.py
|--- base.py
|--- database.db
|--- models.py
|--- queries.py

```

Die Datei „database.db“ stellt die Datenbank dar. Es enthält alle Tabellen und Daten, die eingetragen wurden. Die Abbildung 2 zeigt die in der Datenbank gelegte Tabellen und deren Verknüpfungen.

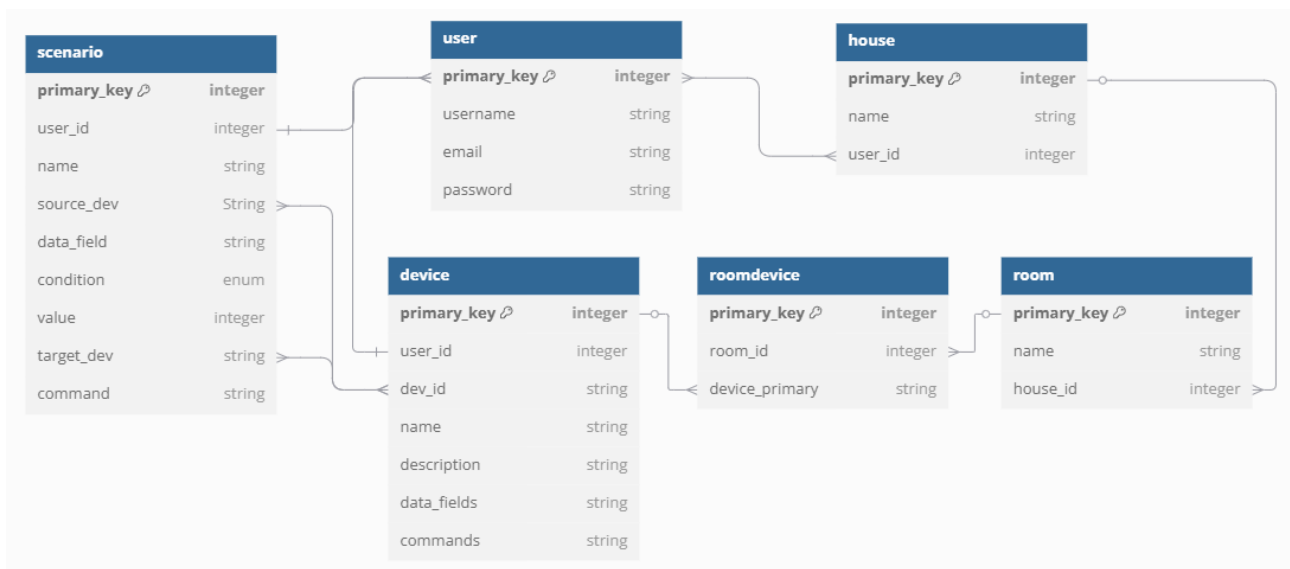


Abbildung 2: Datenbank

- **user** — Enthält die Benutzerdaten.
 - **primary_key**: Ein Primärschlüssel vom Typ Integer, der automatisch inkrementiert wird.
 - **username**: Der Benutzername, der für jeden Benutzer eindeutig ist.
 - **email**: Die E-Mail-Adresse des Benutzers, ebenfalls einzigartig.
 - **password**: Das Passwort des Benutzers, das verschlüsselt gespeichert wird.

Ein Benutzer kann mehrere Häuser und Geräte haben.

- **house** — Speichert die vom Benutzer erstellten Häuser.
 - **primary_key**: Ein Primärschlüssel vom Typ Integer, der automatisch inkrementiert wird.

- **name:** Der vom Benutzer erstellte Name des Hauses.
- **user_id:** Ein Fremdschlüssel, der auf den Primärschlüssel der Tabelle **user** verweist, um anzugeben, welchem Benutzer das Haus gehört.

Ein Haus gehört zu einem bestimmten Benutzer. Ein Haus kann mehrere Räume enthalten.

- **room** — Speichert die vom Benutzer erstellten Räume.

- **primary_key:** Ein Primärschlüssel vom Typ Integer, der automatisch inkrementiert wird.
- **name:** Der vom Benutzer erstellte Name des Raums.
- **house_id:** Ein Fremdschlüssel, der auf den Primärschlüssel der Tabelle **house** verweist.

Ein Raum gehört immer zu einem bestimmten Haus. Ein Raum kann mehrere Geräte enthalten.

- **device** — Speichert die vom Benutzer angemeldeten Geräte.

- **primary_key:** Ein Primärschlüssel vom Typ Integer, der automatisch inkrementiert wird.
- **dev_id:** Die eindeutige, vordefinierte ID des Geräts.
- **name:** Der vom Benutzer erstellte Name des Geräts.
- **user_id:** Ein Fremdschlüssel, der auf den Primärschlüssel der Tabelle **user** verweist und angibt, welchem Benutzer das Gerät gehört.
- **description:** Eine optionale Beschreibung des Geräts.
- **data_fields:** Die Datenfelder, die vom Gerät geliefert und dem Benutzer zur Verfügung gestellt werden. Werden im CSV-Format gespeichert, getrennt durch „，“.
- **commands:** Die Befehle, die das Gerät ausführen kann. Werden im CSV-Format gespeichert, getrennt durch „，“.

Ein Gerät gehört genau einem Benutzer und einem Raum. Ein anderes Gerät mit derselben Geräte-ID kann jedoch von einem anderen Benutzer angemeldet und einem anderen Raum zugeordnet werden.

- **roomdevice** — Speichert die Zuordnung der Geräte zu den Räumen.

- **primary_key:** Ein Primärschlüssel vom Typ Integer, der automatisch inkrementiert wird.
- **room_id:** Ein Fremdschlüssel, der auf den Primärschlüssel der Tabelle **room** verweist.
- **device_primary:** Ein Fremdschlüssel, der auf den Primärschlüssel der Tabelle **device** verweist.

- **scenario:** — Speichert die vom Benutzer erstellten Szenarien.

- **primary_key:** Ein Primärschlüssel vom Typ Integer, der automatisch inkrementiert wird.
- **user_id:** Ein Fremdschlüssel, der auf den Primärschlüssel der Tabelle **user** verweist.

- **name:** Der vom Benutzer erstellte Name des Szenarios
- **source_dev:** Gerät, das das Szenario auslöst. Ein Fremdschlüssel, der auf der Spalte `dev_id` der Tabelle `device` verweist.
- **data_field:** Datenfeld des Quellgeräts.
- **condition:** Bedingung für das Auslösen der Szenario. Mögliche Werte sind „>“, „>“ und „==“. Diese Werte repräsentieren die Bedingungen „größer als“, „kleiner als“ und „gleich“.
- **value:** Der Wert des Datenfeldes.
- **target_dev:** Zielgerät, der ein Befehl beim Auslösen des Szenarios ausführt.
- **command:** Der Befehl, der das Zielgerät ausführt.

Ein Benutzer kann mehrere Szenarios erstellen.

Initialisierung der Datenbank

Beim Import des Moduls „db“ wird der Skript „`__init__`“ aufgerufen (Listing 8), der die Datenbank initialisiert.

Zunächst wird eine asynchrone Datenbank-Engine „`AsyncEngine`“ erstellt, die die Verbindungen zur Datenbank verwaltet (Zeile 7). Als Parameter erhält sie den Pfad zur Datenbank, die im Paket „`config`“ gespeichert ist.

Der „`sessionmaker`“ (Zeile 9-11) ist eine Fabrikfunktion zur Verwaltung von Datenbank-Sessions. Er erhält die Datenbank-Engine sowie zusätzliche Parameter, die das Verhalten der erstellten Sessions steuern. Mit der Funktion `get_session()` (Zeile 13-15) wird eine asynchrone `AsyncSession` erzeugt, die Zugriff auf die Datenbank ermöglicht.

Die Funktion `create_tables()` (Zeile 17-20) initialisiert alle Tabellen in den Klassenmodellen, die von der Klasse `Base` erben. Somit ist die Datenbankverbindung mit `SQLAlchemy` initialisiert.

```

1  from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
2  from sqlalchemy.orm import sessionmaker
3  from app.config import Config
4  from . import models
5  from .base import Base
6
7  async_engine = create_async_engine(Config.SQLALCHEMY_DATABASE_URL)
8
9  async_session = sessionmaker(
10     async_engine, expire_on_commit=True, class_=AsyncSession
11 )
12
13 async def get_session():
14     async with async_session() as session:
15         yield session
16
17 async def create_tables():
18     async with async_engine.begin() as connection:
19         await connection.run_sync(Base.metadata.create_all)
20         await connection.commit()

```

Listing 8: `db:__init__`

Modelle

In der Datei „models.py“ sind die Datenbank-Tabelle als Python-Klassen definiert.

Zunächst müssen die benötigten Module importiert werden. Das Modul „bcrypt“ wird für die Entschlüsselung und Verifizierung der Passwörter verwendet. Die Module aus der Bibliothek „SQLAlchemy“ enthalten die Funktionen für die Interaktionen mit der Datenbank.

Das Modul „.base“ entspricht der Datei „base.py“ (Listing 9). In dieser Datei wird lediglich „declarative_base“ importiert und in der Variablen `Base` initialisiert. Dieser Schritt ist notwendig, um auf die Base in anderen Skripten zugreifen zu können.

```
1 from sqlalchemy.orm import declarative_base
2 Base = declarative_base()
```

Listing 9: db: base.py

Im Listing 10 ist die Tabelle „house“ als Python-Klasse „HouseModel“ definiert. Der Name der Tabelle wird über die Variable „__tablename__“ festgelegt. Die Spalten sind als Objekte der Klasse „Column“ aus der Bibliothek „SQLAlchemy“ definiert, wobei jedem Objekt entsprechende Parameter zugewiesen werden.

In SQLAlchemy muss eine Beziehung zwischen dem `ParentModel` und dem `ChildModel` definiert werden, wenn der Fremdschlüssel des `ChildModel` auf das `ParentModel` verweist. Diese Beziehung dient dazu, Konflikte beim Löschen oder Aktualisieren von Einträgen zu vermeiden.

In der Klasse `HouseModel` wird die Beziehung zwischen der Tabelle „house“ und der Tabelle „room“ durch das Attribut `rooms` (Zeile 10-12) definiert. Diese Beziehung legt fest, dass alle zugehörigen Räume gelöscht werden, wenn das Haus gelöscht wird.

Des Weiteren sorgt die Klassenvariable „__table_args__“ (Zeile 14) dafür, dass die Kombination der Spalten „name“ und „user_id“ eindeutig ist. Damit wird festgelegt, dass die Namen der Häuser eines Benutzers eindeutig sein müssen.

```
1 class HouseModel(Base):
2     __tablename__ = 'house'
3     primary_key = Column(Integer, primary_key=True,
4         autoincrement=True, nullable=False)
5     name = Column(String(50), nullable=False)
6     user_id = Column(
7         Integer, nullable=False,
8         ForeignKey("user.primary_key", ondelete='CASCADE'))
9     rooms = relationship(
10         "RoomModel", backref="house",
11         cascade="all, delete-orphan", lazy='selectin')
12
13     __table_args__ = (UniqueConstraint('name', 'user_id'),)
```

Listing 10: models.py: HouseModel

Die Klasse „UserModel“ (Listing 11) repräsentiert die Tabelle „user“. Neben der Definition der Spalten enthält diese Klasse eine Funktion zur Validierung von Passwörtern. Die Funktion erhält ein Passwort als Parameter und vergleicht es mit dem im Modell gespeicherten und verschlüsselten Passwort. Dadurch muss das gespeicherte Passwort nicht entschlüsselt werden.

```
1 class UserModel(Base):
2     __tablename__ = 'user'
3     def verify_password(self, password:str):
4         return bcrypt.checkpw(password.encode('utf-8'), self.password)
5     # Definition der Spalten ...
```

Listing 11: models.py: UserModel

Die Klasse „RoomModel“ (Listing 12) stellt die Datenbanktabelle „room“ dar.

Diese Klasse enthält eine Beziehung zu der Klasse „RoomDeviceModel“. Dies ist erforderlich, um vom Eintrag in der Tabelle „room_device“ direkt auf das zugehörige Raum-Objekt zugreifen zu können.

Der Parameter „cascade='all, delete-orphan'“ stellt sicher, dass beim Löschen eines Raumes automatisch alle zugehörigen Einträge in der Verknüpfungstabelle gelöscht werden.

Zusätzlich wird durch „back_populates“ die Beziehung bidirektional definiert. Das bedeutet, dass von einem Geräteeintrag auf den zugehörigen Raum zugegriffen werden kann.

Der Parameter „lazy='selectin'“ legt fest, dass verknüpfte Geräte zusammen mit dem Raum geladen werden. Dadurch wird die Anzahl der Datenbankabfragen reduziert.

```
1  class RoomModel(Base):
2      __tablename__ = 'room'
3
4      # Definition der Spalten ...
5
6      devices = relationship("RoomDeviceModel", back_populates="room",
7                             cascade="all, delete-orphan", lazy='selectin')
8      __table_args__ = (UniqueConstraint('name', 'house_id'),)
```

Listing 12: models.py: RoomModel

Die Klasse „DeviceModel“ 13 repräsentiert die Datenbanktabelle „device“, in der alle registrierten Geräte gespeichert werden.

Die Tabelle enthält einen Fremdschlüssel „user_id“, der auf die Tabelle „user“ verweist. Dieser Fremdschlüssel gibt an, welchem Benutzer das jeweilige Gerät zugeordnet ist. Mit dem Parameter „ondelete='CASCADE'“ wird sichergestellt, dass beim Löschen eines Benutzers auch alle zugehörigen Geräteeinträge automatisch entfernt werden.

Die Klassenvariable „__table_args__“ stellt sicher, dass ein Benutzer nicht mehrere Geräte mit der gleichen Geräte-ID registrieren kann.

In Zeile 15 ist die Beziehung zur Klasse „RoomDeviceModel“ definiert. Dies ist notwendig, um von einem Geräteeintrag auf die zugehörigen Verknüpfungen in der Tabelle „room_device“ zugreifen zu können.

Ähnlich wie in der Klasse „RoomModel“ sorgen die Parameter dafür, dass verknüpfte Räume zusammen mit dem Gerät geladen werden und dass beim Löschen eines Geräts auch alle zugehörigen Einträge in der Verknüpfungstabelle automatisch entfernt werden.

Zusätzlich sind in der Klasse „DeviceModel“ die Beziehungen zu der Klasse „ScenarioModel“ definiert. Diese Beziehungen stellen sicher, dass beim Löschen eines Geräts alle zugehörigen Szenarien entfernt werden. Gleichzeitig ermöglichen sie den direkten Zugriff auf die entsprechenden Szenarien aus dem Geräteeintrag heraus.

Insgesamt sorgen diese **relationship**-Definitionen für eine konsistente und wartungsfreundliche Datenbankstruktur. Sie stellen sicher, dass abhängige Objekte automatisch verwaltet werden und verhindern typische Fehler beim Löschen oder Bearbeiten von Einträgen.

Die Klasse „RoomDeviceModel“ (Listing 14) stellt die Datenbanktabelle „roomdevice“ dar. In dieser Tabelle werden die Verknüpfungen zwischen Räumen und Geräten gespeichert.

Die Tabelle enthält zwei Fremdschlüssel „room_id“ und „device_primary“. Der Fremdschlüssel „room_id“ verweist auf die Tabelle „room“. Der Fremdschlüssel „device_primary“ verweist auf die Tabelle „device“.

Der Parameter „ondelete='CASCADE'“ stellt sicher, dass beim Löschen des zugehörigen Raumes oder Gerätes die entsprechende Zuordnung automatisch gelöscht wird.

Die Klasse definiert auch die Beziehungen zu den Klassen „RoomModel“ und „DeviceModel“. Dies ermöglicht den direkten Zugriff auf die entsprechenden Objekte.

Zusätzlich wird in Zeile 14 definiert, dass ein Gerät nicht mehrfach demselben Raum zugeordnet werden kann.

```
1 class DeviceModel(Base):
2     __tablename__ = 'device'
3
4     primary_key = Column(Integer, primary_key=True, autoincrement=True,
5                             nullable=False)
6     dev_id = Column(String(20), nullable=False)
7     name = Column(String(50), nullable=False)
8     user_id = Column(Integer,
9                         ForeignKey('user.primary_key', ondelete='CASCADE'),
10                                nullable=False, unique=False)
11     description = Column(String(250), nullable=True)
12     data_fields = Column(String(256), nullable=True)
13     commands = Column(String(256), nullable=True)
14     __table_args__ = (UniqueConstraint('user_id', 'dev_id'),)
15
16     dev_rooms = relationship("RoomDeviceModel", back_populates="device",
17                             cascade="all, delete-orphan", lazy='selectin')
18
19     scenarios_if = relationship("ScenarioModel",
20                                foreign_keys=[ScenarioModel.source_dev],
21                                back_populates="if_device", cascade="all, delete-orphan",
22                                lazy="selectin")
23     scenarios_then = relationship("ScenarioModel",
24                                  foreign_keys=[ScenarioModel.target_dev],
25                                  back_populates="then_device",
26                                  cascade="all, delete-orphan", lazy="selectin")
```

Listing 13: models.py: DeviceModel

```
1 class RoomDeviceModel(Base):
2     __tablename__ = 'room_device'
3     primary_key = Column(Integer, primary_key=True, autoincrement=True,
4                             nullable=False)
5     room_id = Column(Integer,
6                       ForeignKey('room.primary_key', ondelete='CASCADE'),
7                               nullable=False)
8     device_primary = Column(Integer,
9                              ForeignKey('device.primary_key', ondelete='CASCADE'),
10                                nullable=False)
11
12     room = relationship("RoomModel", back_populates="devices")
13     device = relationship("DeviceModel", back_populates="dev_rooms")
14
15     __table_args__ = (UniqueConstraint('room_id', 'device_primary'),)
```

Listing 14: models.py: RoomDeviceModel

Die Klasse „ScenarioModel“ (Listing 15) repräsentiert die Datenbanktabelle „scenario“, in der benutzerdefinierte Automatisierungsszenarien gespeichert werden.

Der Fremdschlüssel „user_id“ (Zeile 12-14) verknüpft das Szenario mit dem zugehörigen Benutzer. Durch die Angabe von „ondelete='CASCADE'“ wird sichergestellt, dass beim Löschen eines Benutzers auch alle zugehörigen Szenarien automatisch entfernt werden.

Die Spalte „source_dev“ (Zeile 15-17) speichert die Geräte-ID des Quellgeräts. In der Spalte „target_dev“ (Zeile 21-23) wird die Geräte-ID des Zielgeräts gespeichert. Beide Spalten verweisen auf die Spalte „dev_id“ der Tabelle „device“.

In der Spalte „data_field“ (Zeile 18) wird das Datenfeld gespeichert, dessen Wert als Auslöser für das Szenario geprüft wird.

Die Spalte „condition“ (Zeile 19) stellt die Bedingung, die erfüllt sein muss, damit das Szenario ausgelöst wird. Mögliche Vergleichsoperatoren sind in der zugehörigen Enum-Klasse „ScenarioConditionsEnum“ (Zeile 2-5) festgelegt.

Die Beziehungen „if_device“ und „then_device“ verweisen auf die Spalte „device_id“ der Klasse „DeviceModel“. Sie ermöglichen den direkten Zugriff auf die zugehörigen Geräte. In der Klasse „DeviceModel“ sind diese unter den Bezeichnungen „scenarios_if“ und „scenarios_then“ (Listing 13, Zeile 18–25) definiert.

```
1  class ScenarioModel(Base):
2      class ScenarioConditionsEnum(enum.Enum):
3          greater = ">"
4          less = "<"
5          equal = "=="
6
7      __tablename__ = "scenario"
8
9      primary_key = Column(Integer, primary_key=True,
10         autoincrement=True, nullable=False)
11      name = Column(String(20), nullable=False)
12      user_id = Column(Integer,
13         ForeignKey("user.primary_key", ondelete='CASCADE'),
14         nullable=False)
15      source_dev = Column(String(20),
16         ForeignKey("device.dev_id", ondelete='CASCADE'),
17         nullable=False)
18      data_field = Column(String(20), nullable=False)
19      condition = Column(Enum(ScenarioConditionsEnum), nullable=False)
20      value = Column(Integer, nullable=False)
21      target_dev = Column(String(20),
22         ForeignKey("device.dev_id", ondelete='CASCADE'),
23         nullable=False)
24      command = Column(String(50), nullable=False)
25
26      if_device = relationship("DeviceModel",
27         foreign_keys=[source_dev], lazy='selectin')
28      then_device = relationship("DeviceModel",
29         foreign_keys=[target_dev], lazy='selectin')
```

Listing 15: models.py: ScenarioModel

Die beschriebenen Datenbankmodelle bilden ein konsistentes Schema. Die Beziehungen zwischen den Tabellen sind logisch strukturiert und berücksichtigen potenzielle Konflikte bei Datenänderungen. Sie ermöglichen einen effizienten Zugriff auf zusammengehörige Objekte. Mithilfe bidirektionaler Beziehungen sowie automatischer Löschregeln wird die Datenintegrität gewahrt und die Handhabung im Programmcode vereinfacht.

Interaktionen mit der Datenbank

In der Datei „queries.py“ sind die benötigten Funktionen für die Interaktion mit der Datenbank definiert. Diese Funktionen ermöglichen das Einfügen, Löschen, Abrufen und Aktualisieren der Daten in der Datenbank.

Jede Funktion bekommt ein `AsyncSession`-Objekt als erster Parameter. Über dieses Session-Objekt werden die Datenbankoperationen asynchron ausgeführt.

Zur Behandlung möglicher Datenbankfehler enthält jede Funktion einen Try-Catch-Block.

Ein `IntegrityError` wird ausgelöst, wenn eine Operation gegen eine Integritätsbedingung

der Datenbank verstößt. Diese Fehler wird bei Verletzung einer Fremdschlüsselbeziehung, eines Unique Constraints oder beim Einfügen von Werten mit einem falschen Typ erzeugt.

Ein `SQLAlchemyError` umfasst alle Fehler, die von SQLAlchemy ausgelöst werden, wenn die Operation nicht erfolgreich war. Dieser Fehler wird bei Syntaxfehlern in der SQL-Abfrage oder bei Fehlern in der Datenbankverbindung ausgelöst.

Alle sonstigen Fehler, die nicht durch SQLAlchemy verursacht sind, werden in einem `Exception`-Block behandelt.

Bei jedem Fehler wird der Fehler geloggt und die Session zurückgesetzt, damit keine unvollständigen Daten in der Datenbank gespeichert werden. Anschließend wird eine `HTTPException` mit Status und Beschreibung des Fehlers zurückgegeben.

Diese ist eine standardisierte Behandlung der Fehler in allen Funktionen, die Datenbankoperationen ausführen.

Als Beispiel wird im Folgenden die Struktur der Funktion `add_user()` näher erläutert. Diese Funktion fügt einen neuen Benutzer in die Datenbank ein.

Die Funktion erhält als Parameter ein `AsyncSession`-Objekt sowie die Benutzerdaten: Benutzername, Email und Passwort.

Zunächst wird das Passwort verschlüsselt und in der Variable „`password_hash`“ gespeichert (Zeile 3-4).

Anschließend wird ein Objekt „`new_user`“ der Klasse `UserModel` mit den übergebenen Parametern erzeugt (Zeile 5-6). Dieses Objekt wird der Session hinzugefügt (Zeile 7). Danach werden die Änderungen in der Datenbank gespeichert (Zeile 8). Anschließend wird das `new_user`-Objekt aktualisiert (Zeile 9), um den generiert Primärschlüssel zu bekommen. Dieser wird zurückgegeben (Zeile 10).

Existiert der übergebene Benutzername oder die E-Mail-Adresse bereits in der Datenbank, wird ein `IntegrityError` ausgelöst, und die entsprechende Fehlermeldung wird zurückgegeben.

Tritt ein Fehler bei der Ausführung der SQL-Abfrage auf so wird ein `SQLAlchemyError` ausgelöst.

Alle sonstigen Fehler, die nicht direkt durch SQLAlchemy verursacht werden, werden durch den `Exception`-Block behandelt.

```
1 async def add_user(db_session: AsyncSession, username: str, email: str,
2     password: str):
3     try:
4         password_hash = bcrypt.hashpw(password.encode('utf-8'),
5             bcrypt.gensalt())
6         new_user = UserModel(username=username, email=email,
7             password=password_hash)
8         db_session.add(new_user)
9         await db_session.commit()
10        await db_session.refresh(new_user)
11        return new_user.primary_key
12    except IntegrityError as e:
13        _logger.error(f"IntegrityError: {e}")
14        await db_session.rollback()
15        raise HTTPException(status_code=422, detail="ALREADY EXISTS")
16    except SQLAlchemyError as e:
17        _logger.error(f"SQLAlchemyError: {e}")
18        await db_session.rollback()
19        raise HTTPException(status_code=500, detail="DATABASE ERROR")
20    except Exception as e:
21        _logger.error(f"Exception: {e}")
22        await db_session.rollback()
23        raise HTTPException(status_code=500, detail="UNEXPECTED ERROR")
```

Listing 16: queries.py: add_user

5.2 Webbasierte Schnittstelle (API)

Um die Benutzerinteraktionen mit dem System zu ermöglichen, muss eine Schnittstelle gebaut werden. Diese Schnittstelle muss den gesicherten Zugriff zu dem System bieten, die notwendigen Informationen bereitstellen, auf die Benutzerinteraktionen reagieren und die Daten verwalten. Nach sorgfältiger Abwägung verschiedener Optionen habe ich mich für die Implementierung einer webbasierten Schnittstelle auf Basis von **FastAPI** entschieden.

Die zentrale Idee hinter der Entwicklung der API besteht darin, dem Benutzer eine effiziente und sichere Möglichkeit zu bieten, IoT-Geräte in einer strukturierten Umgebung zu verwalten. Dies umfasst das Hinzufügen neuer Geräte, die Gruppierung dieser Geräte in Räumen sowie die Verwaltung mehrerer Räume innerhalb eines Hauses. Darüber hinaus ermöglicht die API die kontinuierliche Überwachung und Steuerung der hinzugefügten Geräte.

Der Webserver ist, wie auch anderen Komponenten, in einem Paket namens „**webserver**“ strukturiert. Die Struktur des Packets ist wie folgt aufgebaut:

```
bachelor/code/spa/webserver
|
|--- __init__.py
|--- routes.py
|--- services.py
|--- templates/
|--- static/
```

Die Datei „**__init__.py**“ dient zur Initialisierung der Webanwendung. Zunächst wird eine Instanz der Klasse **FastAPI** erzeugt, die als zentrales Anwendungsobjekt dient. Anschließend wird das statische Verzeichnis eingebunden, sodass HTML-, CSS- und JavaScript-Dateien direkt vom Server ausgeliefert werden können. Abschließend wird der **Router** eingebunden, über den die definierten HTTP-Routen der Anwendung bereitgestellt werden.

In der Datei „**routes.py**“ sind alle HTTP-Routen definiert. Diese werden in einem Router gesammelt und an das zentrale Anwendungsobjekt angebunden.

Die Datei „**services.py**“ enthält die Funktionen zur Verarbeitung der HTTP-Anfragen, die in der Datei „**routes.py**“ aufgerufen werden. Außerdem sind in der Datei die **Pydantic**-Modelle sowie die Funktionen zur Verwaltung des WebSockets implementiert.

Im Ordner „**templates**“ befinden sich die HTML-Dateien, die von dem Server ausgeliefert werden. Der Ordner „**static**“ enthält die Styling-Dateien und der Unterordner „**js**“ das JavaScript-Skript zur dynamische Erstellung der Weboberfläche.

5.2.1 Sicherheit

Sicherheit spielt in diesem System eine wichtige Rolle. Für die API gibt es zwei kritische Sicherheitspunkte: die Übertragung sensibler Daten und die Authentifizierung der Benutzer.

Um sensible Daten, wie Passwörter, sicher zu speichern, wird der Algorithmus „**bcrypt**“ verwendet. Anstatt Passwörter im Klartext zu speichern, wird bei der Registrierung eines Benutzers das Passwort gehasht. Der Hash wird in der Datenbank gespeichert, nicht das Passwort selbst. Bei der Anmeldung wird das eingegebene Passwort mit dem gespeicherten Hash verglichen, ohne dass das Originalpasswort jemals entschlüsselt wird. Dadurch sind die Benutzerdaten auch dann geschützt, wenn die Datenbank kompromittiert wird.

Zur Authentifizierung und Identifizierung des Benutzers nach erfolgreicher Anmeldung wird das JWT-Verfahren⁵ verwendet. Die Funktion `create_jwt_token()` in der Datei „**services.py**“ generiert ein JWT-Token, das an den Benutzer zurückgegeben wird. Dieses Token enthält die

⁵JSON Web Token

Benutzer-ID in verschlüsselter Form, besitzt eine in der Konfigurationsdatei definierte Ablaufzeit und wird mit einem geheimen Schlüssel signiert.

Das Token muss clientseitig gespeichert und bei jeder Anfrage im HTTP-Header mitgesendet werden, um die Authentifizierung des Benutzers sicherzustellen. Der entsprechende Header muss dabei die folgende Struktur aufweisen:

```
headers: {  
    "auth": "Bearer <JWT TOKEN>"  
}
```

Das Request-Objekt wird an die Funktion `get_token()` in der Datei „`routes.py`“ übergeben, die das JWT-Token aus dem HTTP-Header extrahiert. Dabei wird geprüft, ob der Header das Feld „`auth`“ enthält und ob dessen Wert mit dem Schlüsselwort „`Bearer`“ beginnt. Ist dies nicht der Fall, wird eine Exception ausgelöst, da das Token in einem ungültigen Format vorliegt. Im normalen Ablauf wird das Token aus dem Header extrahiert und zur weiteren Verarbeitung zurückgegeben.

Die Verifizierung des Tokens stellt eine grundlegende Voraussetzung für alle API-Funktionen dar. Nur gültige und nicht abgelaufene Tokens ermöglichen den Zugriff auf geschützte Ressourcen der API.

Das erhaltene Token wird an die Funktion `verify_token()` in der Datei „`services.py`“ übergeben, um dessen Gültigkeit zu prüfen. Zunächst wird das Token entschlüsselt, und die darin enthaltene Benutzer-ID wird extrahiert.

Ist das Token abgelaufen, ungültig, fehlerhaft oder unvollständig, wird eine entsprechende Fehlermeldung ausgegeben und eine HTTP-Exception mit dem entsprechenden Statuscode ausgelöst. Andernfalls wird die Benutzer-ID für nachfolgende Verarbeitungsschritte zur Verfügung gestellt.

Anmeldung

Damit ein Benutzer die Funktionen der API nutzen kann, muss zunächst ein Konto erstellt werden. Dies erfolgt durch Senden eines POST-Requests an die Route „`/sign_up`“. Der Request muss die erforderlichen Parameter enthalten: `username`, `email` und `password`. Nach Erhalt der Anfrage wird die Funktion `signup_post()` aufgerufen, die die eingegebenen Daten verarbeitet und die Benutzerregistrierung durchführt.

Die Registrierungsdaten werden als `SignUpModel`-Objekt entgegengenommen und an die Funktion `services.signup_user()` weitergeleitet. Diese Funktion übernimmt die Validierung der Eingaben, indem sie prüft, ob alle erforderlichen Werten übergeben wurden. Wenn ein Wert fehlt, wird die entsprechende Fehlermeldung geloggt und zurückgegeben.

Sind alle Daten vollständig, wird der Benutzer mit der Funktion `queries.add_user()` in der Datenbank gespeichert. Diese Funktion gibt ein Fehler zurück, wenn der Benutzername oder die Email schon in der Datenbank gespeichert sind.

Nach erfolgreicher Speicherung wird ein JWT-Token generiert und zurückgegeben. Dieses Token enthält die Benutzer-ID und hat eine begrenzte Lebensdauer. Mit diesem Token kann der Benutzer direkt angemeldet werden. Tritt während des Prozesses eine Exception auf, so wird diese geloggt und eine entsprechende Fehlermeldung ausgegeben.

Einloggen

Um einem bereits registrierten Benutzer den Zugriff auf die API zu ermöglichen, ist eine Authentifizierung erforderlich. Diese erfolgt durch das Senden eines POST-Requests an die Route „`/login`“. Der Request muss die Parameter `username` und `password` enthalten. Nach Erhalt der

Anfrage wird die Funktion `login_post()` aufgerufen, die die Authentifizierung des Benutzers vornimmt.

Die Benutzerdaten werden als `UserLoginModel` entgegengenommen und an die Funktion `services.auth_user()` übergeben. Diese Funktion überprüft, ob die erforderlichen Werte `username` und `password` übergeben wurden. Ist es nicht der Fall, wird eine entsprechende Fehlermeldung ausgegeben.

Anschließend wird mit der Funktion `queries.get_user_data()` geprüft, ob ein Benutzer mit dem angegebenen `username` in der Datenbank existiert. Wird kein entsprechender Datensatz gefunden, wird eine `HTTPException` mit dem Statuscode 401 und der Meldung „Invalid username“ ausgelöst.

Wenn der Benutzer existiert, wird das eingegebene Passwort mit der gespeicherten verschlüsselten Version verglichen. Ist die Prüfung erfolgreich, gibt die Funktion den Primärschlüssel zurück. Andernfalls wird die Fehlermeldung „Wrong password“ zurückgegeben.

Nach erfolgreicher Authentifizierung wird in der Funktion `login_post()` ein JWT-Token erzeugt und zurückgegeben. Tritt während des Prozesses eine Exception auf, wird diese protokolliert und eine entsprechende Fehlermeldung ausgegeben.

Nach dem erfolgreichen Eingloggen und Erhalt des Tokens, kann der Benutzer zu den Funktionen der API zugreifen.

5.2.2 API-Endpunkte

In der Datei „`routes.py`“ sind die API-Endpunkte definiert, die verschiedene Funktionen des Systems bereitstellen. Jeder Endpunkt ist dabei einer bestimmten Aufgabe zugeordnet.

Zur besseren Strukturierung und Modularisierung des Codes wurden die Hilfsfunktionen in das Skript `services.py` ausgelagert. Diese Funktionen enthalten die Anwendungslogik, verarbeiten die Daten und stellen sie zur Verfügung, verwalten die Verbindungen zum WebSocket.

Nach dem Import der erforderlichen Module wird ein Router mit `APIRouter()` erstellt, der die API-Endpunkte verwaltet. Die API-Endpunkte werden am Router registriert und anschließend mit der Funktion `include_router()` in das Anwendungsobjekt angebunden. Durch die Verwendung von FastAPI-Routern und einer klar strukturierten Modulanordnung bleibt die API übersichtlich, leicht wartbar und erweiterbar.

Um auf Anfragen mit HTML-Dateien zu antworten, wird ein `Jinja2Templates`-Objekt erstellt. Dabei wird der Pfad zum Ordner „`templates/`“ angegeben, der die HTML-Vorlagen enthält. Dieses Objekt ermöglicht es, dynamische HTML-Antworten zu generieren, indem Daten aus den Endpunkten in die Vorlagen eingebunden werden.

Jede Funktion, die auf die Datenbank zugreift, benötigt als Parameter eine Instanz der Klasse `AsyncSession`. Im der Datei „`routes.py`“ wird dazu die Funktion `get_session()` aus dem Paket „`db`“ verwendet, die für die Erzeugung der Datenbanksitzungen zuständig ist.

Durch die Verwendung des Abhängigkeitsmechanismus `Depends(get_session)` wird bei jedem Aufruf einer Router-Funktion automatisch eine neue Datenbanksitzung erzeugt und an die entsprechenden Unterfunktionen weitergegeben.

Dies ermöglicht eine effiziente und asynchrone Kommunikation mit der Datenbank, wodurch mehrere Anfragen gleichzeitig ohne blockierende Wartezeiten bearbeitet werden können.

Im Listing 17 ist ein API-Endpunkt „`/get_user_data`“ definiert, der an dem `router`-Objekt registriert ist (Zeile 1). Beim Empfang einer GET-Anfrage auf dieser Route wird die asynchrone Funktion `get_user_data()` ausgeführt.

Die Funktion erwartet drei Parameter:

- **request:** `Request` – Das Anfrageobjekt, das die vollständige HTTP-Anfrage des Clients enthält.

- `token: str = Depends(get_token)` – Ein Authentifizierungstoken, das über die FastAPI-Abhängigkeitsfunktion `Depends()` aus der Funktion `get_token()` abgeleitet wird.
- `db_session: AsyncSession = Depends(get_session)` – Eine asynchrone Datenbank-Session, die über die Funktion `get_session()` erzeugt wird.

```

1  @router.get('/get_user_data')
2  async def get_user_data(
3      request: Request,
4      token: str = Depends(get_token),
5      db_session: AsyncSession = Depends(get_session)
6  ):
7      try:
8          user_id = services.verify_token(token)
9          if not user_id:
10             return {'error': 'TOKEN INVALID'}
11         res = await services.get_user_data(db_session, user_id)
12         return JSONResponse(res)
13     except Exception as e:
14         return {'error': e}

```

Listing 17: routes.py: get_user_data

Nach dem Empfang der Anfrage wird innerhalb eines `try`-Blocks die Benutzer-ID mithilfe der Funktion `services.verify_token()` aus dem erhaltenen Token ermittelt (Zeile 8–10). Detaillierter wurde dieser Vorgang in Abschnitt 5.2.1 erklärt.

Anschließend wird die asynchrone Funktion `services.get_user_data()` aufgerufen, die als Parameter die Datenbank-Session sowie die Benutzer-ID erhält (Zeile 11). Diese Funktion liefert die in der Datenbank gespeicherten Benutzerdaten im JSON-Format. Das erhaltene JSON-Objekt wird mit `JSONResponse()` an den Client zurückgegeben.

Dies stellt den Standardvorgang zur Verarbeitung von HTTP-Anfragen in FastAPI dar. Die HTTP-Anfragen werden mit entsprechenden Funktionen verknüpft. Nach der Bearbeitung der Anfrage wird eine Antwort an dem Client geliefert. Im Fehlerfall wird das Fehler an den Client zurückgegeben.

Haus und Raum

Die Benutzer können ihre Geräte nach Räumen und die Räume nach Häusern gruppieren. Dies erleichtert die Verwaltung und Überwachung der Geräte. Besonders im Frontend ist dies sehr nützlich, da es eine klare Strukturierung des Hauses geschaffen wird. Dabei kann ein Benutzer mehrere Häuser besitzen.

Um ein Haus zu anzulegen, muss ein `POST`-Request an die Route „`add_house`“ gesendet werden. Der Request muss im Header ein Bearer-Token und im Body den Namen des Hauses enthalten. Nach Erhalt des Requests wird die Funktion `add_house_post()` aufgerufen. Standardmäßig wird das Token validiert, und die Benutzer-ID wird aus dem Token extrahiert, um das Haus dem entsprechenden Benutzer zuzuordnen. Wenn eine gültige Benutzer-ID aus dem Token erzeugt wurde, wird diese zusammen mit der Instanz der Klasse `AsyncSession` und dem Namen des Hauses an die Funktion `services.create_new_house()` übergeben.

Diese Funktion prüft zunächst, ob ein gültiger Hausname übergeben wurde. Anschließend wird mit der Funktion `queries.add_new_house()` ein neuer Eintrag in der Datenbank angelegt und dem Benutzer zugeordnet.

Wenn die Erstellung des Hauses erfolgreich war, gibt die Funktion eine Bestätigung zurück. Im Fehlerfall wird eine entsprechende Fehlermeldung generiert und geloggt.

Das Verfahren zum Hinzufügen eines Raumes zu einem House ist im Wesentlichen dasselbe wie bei der Erstellung eines Houses. An die Route „`add_room`“ wird ein `POST`-Request mit dem

Token im Header und mit dem Namen des Raumes sowie der zugehörigen House-ID im Body gesendet. Auch hier wird das JWT-Token standardmäßig validiert, und die Benutzer-ID daraus extrahiert.

Die Daten des Raumes und die Benutzer-ID werden an die Funktion `services.add_room()` übergeben. Diese Funktion holt zunächst mit dem Befehl `queries.get_house()` die Daten des Hauses aus der Datenbank, um den Besitzer des Hauses zu bestätigen.

Stimmt die übergebene Benutzer-ID mit der in der Datenbank gespeicherten Benutzer-ID des Hausbesitzers überein, wird mit der Funktion `queries.add_new_room()` das neue Raum in der Datenbank gespeichert. Im Erfolgsfall wird eine Bestätigung zurückgegeben.

Wenn der Benutzer nicht berechtigt ist, einen Raum zu dem angegebenen Haus hinzuzufügen, gibt die Funktion eine Meldung über unberechtigten Zugriff zurück.

Der Benutzer hat die Möglichkeit seine Häuser und Räume zu löschen.

Beim Löschen eines Hauses wird ein Request mit den Benutzer- und House-ID an die Route „`/delete_house`“ gesendet. Nach der Validierung des Tokens und Erzeugung der Benutzer-ID werden die Daten an die Funktion `services.delete_house()` übergeben. Diese Funktion prüft, ob das Haus dem Benutzer gehört und entfernt mit der Funktion `queries.delete_house()` den zugehörigen Eintrag aus der Datenbank. Die zugehörigen Räume werden ebenfalls gelöscht. Im Fehlerfall wird der entsprechende Fehler geloggt und ausgegeben.

Das Löschen eines Raumes ist ähnlich dem Löschen eines Hauses. Ein Delete-Request mit der Raum- und Haus-ID wird an die Route „`delete_room`“ gesendet. Nach der Validierung des Tokens und Erzeugung der Benutzer-ID werden die Daten an die Funktion `services.delete_room()` übergeben. Wenn der Benutzer der Eigentümer des Hauses mit der übergebenen House-ID ist, wird der Raum aus der Datenbank gelöscht. Tritt ein Fehler auf, wird dieser geloggt und die entsprechende Meldung wird ausgegeben.

Die Route „`/get_houses`“ gibt alle von Benutzer erstellten Häuser im JSON-Format zurück. Der Request muss im Header nur das Token enthalten, aus dem die Benutzer-ID erzeugt wird. Die Benutzer-ID wird an die Funktion `services.get_houses()` übergeben. Die Funktion `queries.get_houses_on_user()` gibt eine Liste von Objekten der Klasse `HouseModel` zurück, die dem jeweiligen Benutzer zugeordnet sind. Diese Liste wird anschließend in das JSON-Format umgewandelt, um eine strukturierte Darstellung der Daten für die weitere Verarbeitung zu ermöglichen.

Das erstellte Objekt hat folgende Struktur:

```
{
  "house1": {
    "id": 1,
    "name": Mein Haus,
    "rooms": [
      {
        "id": 1,
        "name": Wohnzimmer,
        "devices": [
          DeviceModel
        ]
      }
    ]
  }
}
```

Anschließend wird das erstellte Objekt als `JSONResponse` zurückgegeben.

Device

Die zentrale Aufgabe der API ist die Überwachung und Steuerung der angeschlossenen Geräte. Damit dieses System genutzt werden kann, muss der Benutzer die Möglichkeit haben, seine Geräte in das System einzubinden und zu verwalten.

Jedes Gerät verfügt über eine eindeutige Geräte-ID, die im Programmcode des Gerätes fest hinterlegt ist und zur Identifikation innerhalb des Systems dient. Die Geräte werden in der Datenbank in der Tabelle „device“ als `DeviceModel`-Objekten gespeichert.

Das Hinzufügen eines neuen Gerätes erfolgt durch das Senden eines `POST`-Requests an die Route „/add_new_device“. Der Header des Requests muss ein gültiges JWT-Token enthalten, aus dem die Benutzer-ID ausgelesen wird. Im Body des Requests muss die eindeutige Geräte-ID und der von Benutzer erstellte Name des Gerätes angegeben werden. Der Name dient ausschließlich der Benutzerfreundlichkeit, indem er eine eindeutige Identifikation des Gerätes ermöglicht.

Optional kann ein vom Benutzer erstellter Name und eine Beschreibung des Gerätes sowie die Raum-ID für die Zuordnung des Gerätes zum Raum übergeben werden.

Nach Erhalt des Requests wird die Funktion `add_new_device_post()` aufgerufen. Zuerst wird das Token validiert und die Benutzer-ID aus dem Token erzeugt. Anschließend werden die Gerätedaten an die Funktion `services.add_new_device()` übergeben. Diese prüft zunächst, ob sowohl die Geräte-ID als auch der Gerätename übergeben sind. Fehlt eine dieser Angaben, wird der Vorgang abgebrochen und eine Fehlermeldung zurückgegeben.

Zusätzlich hat der Benutzer die Möglichkeit, das Gerät direkt einem bestimmten Raum innerhalb des Hauses zuzuordnen. Dazu muss im Body des Requests die entsprechende Raum-ID angegeben werden. In diesem Fall wird mit der Funktion `queries.get_house_by_room()` nach dem entsprechenden Haus gesucht. Anschließend wird geprüft, ob die aus dem Token extrahierte Benutzer-ID mit der des Hauseigentümers übereinstimmt. Ist dies nicht der Fall, wird der Zugriff verweigert, der Vorgang abgebrochen und eine entsprechende Fehlermeldung zurückgegeben.

Anschließend wird mit dem Befehl `queries.add_new_device()` das Gerät zunächst ohne direkte Raumzuordnung in der Datenbank gespeichert. Ist eine Raum-ID vorhanden, wird mit dem Befehl `queries.add_room_device()` ein Eintrag in der Tabelle „roomdevice“ erstellt, der den Primärschlüssel des Gerätes mit der Raum-ID verknüpft.

Um ein Gerät aus der Datenbank zu Löschen, muss ein `Delete`-Request mit dem JWT-Token im Header und mit der Geräte-ID im Body auf die Route „/delete_device“ gesendet werden. Die aus dem Token erzeugte Benutzer-ID wird zusammen mit der Geräte-ID an die Funktion `services.delete_device()` übergeben. Diese Funktion prüft zunächst, ob das übergebene Geräte-ID dem Benutzer zugeordnet ist. Im Erfolgsfall wird das Gerät aus der Datenbank gelöscht.

Diese strukturierte Vorgehensweise gewährleistet nicht nur eine sichere und konsistente Registrierung neuer Geräte, sondern bietet dem Benutzer auch die Möglichkeit, seine Geräte flexibel zu verwalten und eindeutig zu identifizieren.

5.3 Szenario

Der Benutzer hat die Möglichkeit, eigene Szenarien zu definieren. Ein Szenario bezeichnet die automatische Ausführung eines Befehls auf einem Zielgerät, sobald die im Szenario definierte Bedingung erfüllt ist.

Ein Szenario besteht aus folgenden Elementen:

- **source_device:** — Das Quellgerät, das die Daten liefert, die zur Prüfung der Bedingung verwendet werden.

- **data_field:** — Das spezifische Datenfeld, das die relevanten Informationen enthält, die für die Bedingung geprüft werden.
- **condition:** — Die Bedingung, die auf das Datenfeld angewendet wird. Es kann größer, kleiner oder gleich sein.
- **value:** — Der Wert, gegen den die Bedingung geprüft wird.
- **target_device:** — Das Zielgerät, auf dem die Aktion ausgeführt wird, wenn die Bedingung erfüllt ist.
- **command:** — Der Befehl, der an das Zielgerät gesendet wird, wenn die Bedingung erfüllt ist

Die Logik eines Szenarios kann schematisch dargestellt werden:

```
IF: source_device.data_field CONDITION value IS TRUE
THEN: target_device DO command
```

Die im Szenario definierte Bedingung wird beim Empfang der Daten vom Quellgerät geprüft. Die Daten aus dem angegebenen Datenfeld werden mit dem in **value** gespeicherten Wert unter Verwendung des in **condition** definierten Vergleichsoperators verglichen. Ist die Bedingung erfüllt, wird der Steuerbefehl **command** an das Zielgerät gesendet.

Die Erstellung eines neuen Szenarios erfolgt durch das Senden eines POST-Requests an die Route „**add_scenario**“. Der Request-Header muss ein gültiges JWT-Token enthalten, während der Request-Body die in der Klasse **services.ScenarioModel** definierten Daten liefern muss.

Nach Erhalt des Requests wird die Funktion **add_scenario()** aufgerufen. Zuerst wird das Token validiert und die Benutzer-ID wird aus dem Token erzeugt. Anschließend wird die Benutzer-ID zusammen mit den Daten des Szenarios an die Funktion **services.add_scenario()** übergeben.

Diese Funktion prüft zunächst, ob sowohl das übergebene Quellgerät (**source_dev**) als auch das Zielgerät (**target_dev**) dem Benutzer zugeordnet sind. Ist dies nicht der Fall, so wird der Vorgang abgebrochen und eine Meldung über den unberechtigten Zugriff zurückgegeben.

Nach erfolgreicher Prüfung werden die Daten des Szenarios an die Funktion **add_scenario()** der Klasse „**queries**“ übergeben, die das neue Szenario in der Datenbank speichert. Abschließend wird mit dem Befehl **MQTTClient.load_scenarios()** die Liste der gespeicherten Szenarien in **MQTTClient** aktualisiert, so dass das System sofort auf die neue Automatisierungsregel reagieren kann.

Das Löschen eines Szenarios erfolgt durch das Senden eines Delete-Requests an die Route „**/delete_scenario**“. Der Request muss im Header ein JWT-Token und im Body das Primärschlüssel des Szenarios enthalten. Zuerst wird die Benutzer-ID aus dem Token extrahiert und zusammen mit dem Primärschlüssel des Szenarios an die Funktion **services.delete_scenario()** übergeben.

Diese Funktion prüft, ob die übergebene Szenario-ID dem Benutzer zugeordnet ist. Im Erfolgsfall wird das Gerät aus der Datenbank gelöscht. Sonst wird eine Meldung über den unberechtigten Zugriff zurückgegeben.

Durch die Erstellung von Szenarien kann der Benutzer sein Haus automatisieren. Ein Szenario wird automatisch ausgelöst, sobald die darin definierte Bedingung erfüllt ist. Dabei kann ein Gerät sowohl als Quellgerät als auch als Zielgerät fungieren. Außerdem kann ein einzelnes Gerät gleichzeitig in mehreren Szenarien beteiligt sein.

5.4 MQTT-Client

Die Geräte senden kontinuierlich ihre Daten über die MQTT-Verbindung an den MQTT-Broker. Jedes Gerät nutzt sein separates Topic, das sich auf die Geräte-ID bezieht. Um die Daten eines bestimmten Geräts zu empfangen, muss ein MQTT-Client das entsprechende Topic abonnieren.

Der MQTT-Client für den Webserver ist im Paket „`mqtt`“ enthalten. In der Datei „`client.py`“ ist die Klasse `MQTTClient` implementiert, die eine asynchrone Schnittstelle zur Verarbeitung von MQTT-Nachrichten bereitstellt.

Die Implementierung basiert auf der Bibliothek `aiomqtt`. Durch die asynchrone Architektur kann der Client effizient auf die eingehenden MQTT-Nachrichten reagieren, ohne blockierende Operationen auszuführen.

Die asynchrone Funktion `start_client()` (Listing 18) wird beim Start des Webservers als asynchrone Aufgabe gestartet. Diese Funktion enthält eine Endlosschleife. Als Parameter erwartet sie eine Liste von Topics, die der Client abonnieren soll.

In der Zeile 13 ist die Funktion `startup()` aus der Datei „`routes.py`“ dargestellt, die beim Start des Webservers aufgerufen wird. In dieser Funktion wird zunächst ein Objekt der Klasse `MQTTClient` erstellt. Danach werden die gespeicherten Szenarien geladen. Anschließend wird eine asynchrone Aufgabe gestartet, die die Funktion `start_client()` der Klasse `MQTTClient` ausführt.

Dadurch wird sichergestellt, dass die MQTT-Verbindung parallel zu anderen asynchronen Aufgaben läuft. Zudem stellt die Endlosschleife sicher, dass die Verbindung nach einer Unterbrechung oder einem Fehler automatisch wiederhergestellt wird.

Innerhalb der Endlosschleife wird zunächst die Verbindung zu dem MQTT-Broker aufgebaut (Zeile 4). Das Objekt `Client` der Klasse `aiomqtt` erwartet als Parameter die IP-Adresse des MQTT-Brokers und den Port. Anschließend werden die übergebenen Topics abonniert (Zeile 5-6).

Danach empfängt der Client die MQTT-Nachrichten asynchron über die `async for`-Schleife, die die eingehenden Nachrichten in `client.messages` verarbeitet. (Zeile 7-8). Jede eingehende Nachricht wird an die Funktion `process_message()` weitergeleitet, die für den Inhalt der Nachricht verarbeitet.

Wenn ein MQTT-Fehler auftritt, wird diese geloggt, aber die Funktion wird nicht abgestürzt.

```
1  async def start_client(cls, topics: list):
2      while True:
3          try:
4              async with aiomqtt.Client(hostname=Config.MQTT_BROKER_ADDRESS,
5                                     port=Config.MQTT_PORT) as client:
6                  for t in topics:
7                      await client.subscribe(t)
8                  async for message in client.messages:
9                      await cls.process_message(message)
10             except aiomqtt.MqttError as e:
11                 logger.error(e)
12
13  # routes.py:
14  async def startup():
15      mqtt_client = MQTTClient()
16      await mqtt_client.load_scenarios()
17      asyncio.create_task(mqtt_client.start_client(TOPIC_LIST))
```

Listing 18: `client.py: start_client`

Die Funktion `process_message()` (Listing 19) extrahiert zunächst das Haupttopic und die Geräte-ID aus dem Attribut „`topic`“ des übergebenen Nachrichtenobjekts (Zeile 3). Anschließend wird die Payload, nämlich der Text, der Nachricht dekodiert und in ein JSON-Objekt

„data“ umgewandelt (Zeile 4-5).

```
1  async def process_message(cls, message):
2
3      main_topic, device_id = str(message.topic).split('/')
4      msg = message.payload.decode()
5      data = json.loads(msg)
6
7      # Handshake
8      if main_topic == 'handshake':
9          from app.db import queries, get_direct_session, close_session
10         session = await get_direct_session()
11         await queries.update_device_handshake(session, device_id, data)
12         await close_session(session)
13
14         # Szenario
15         scenarios = cls.saved_scenarios.get(device_id)
16         if scenarios is not None:
17             for sc in scenarios:
18                 if eval(f"{data[sc.data_field]} {sc.condition.value} {sc.value}")
19                     :
20                     if not sc.active: # scenario is not active
21                         sc.active = True
22                         await cls.send_command_to_device(sc.target_dev, sc.command)
23                     elif sc.active: sc.active = False
24
25         from app.webserver.services import WebsocketHandler
26         await WebsocketHandler.send_data(device_id, data)
```

Listing 19: cleint.py: process_message

Handshake

Die Handshake-Nachricht wird an ein Gerät gesendet, um dessen Konfigurationsdaten zu aktualisieren. Die Klasse MQTT-Client enthält die Funktion `send_handshake()`, die die Geräte-ID als Parameter erwartet und eine MQTT-Nachricht in das Topic „command/Geräte-ID“ mit dem Text „handshake“ sendet. Das Gerät antwortet im Topic „handshake/Geräte-ID“.

Beim Empfang einer MQTT-Nachricht aus dem Topic „handshake“ wird der Inhalt der Nachricht in der Funktion `process_message()` entsprechend verarbeitet (Zeile 8-12). Das JSON-Objekt „data“ enthält die Konfigurationsdaten des Geräts. Die in der Datenbank gespeicherten Datensätze müssen mit diesen Daten aktualisiert werden.

Dazu werden die benötigten Funktionen dynamisch aus dem Paket „db“ importiert. Die erhaltenen Konfigurationsdaten werden zusammen mit der Geräte-ID und der Session an die Funktion `update_device_handshake()` aus dem Skript „queries.py“ übergeben, die alle Datensätze mit gegebener Geräte-ID aktualisiert.

Dieser Vorgang kann jederzeit ausgeführt werden. Damit ist sichergestellt, dass die Daten bei Bedarf aktualisiert werden können.

Szenario

Damit die Szenarien ausgeführt werden können, müssen sie aus der Datenbank geladen und lokal gespeichert werden.

Die Funktion `load_scenarios()` ruft zunächst die Methode `get_all_scenarios()` aus dem Skript „queries.py“ auf, die alle Szenarien zurückgibt. Jedes geladene Szenario wird um ein Attribut „active“ erweitert, das initial auf „False“ gesetzt wird.

Anschließend werden die Szenarien-Objekten als Schlüssel-Wert-Paare in der Klassenvariablen `saved_scenarios` gespeichert. Die Schlüssel entsprechen der Geräte-ID, während die Werte

Listen von Szenarien enthalten, die die Geräte-ID als `source_dev` haben. Existiert bereits ein Eintrag für eine Geräte-ID, wird das Szenario der entsprechenden Liste hinzugefügt, andernfalls wird eine neue Liste erstellt. Die Variable `saved_scenarios` ist wie folgt strukturiert:

```
saved_scenarios = {
    "device1": [ScenarioModel(source_dev == "device1")],
    "device2": [
        ScenarioModel(source_dev == "device2"),
        ScenarioModel(source_dev == "device2")
    ]
}
```

Beim Empfang einer MQTT-Nachricht wird in der Funktion `process_message()` zunächst geprüft, ob für die empfangende Geräte-ID ein Eintrag in der Klassenvariablen `saved_scenarios` vorhanden ist (Zeile 15-16). Ist dies der Fall, werden alle zugehörigen Szenarien iteriert und deren jeweilige Bedingung überprüft (Zeile 17-18).

Die Bedingung wird mit der Funktion `eval` überprüft (Zeile 18). Wenn die Bedingung erfüllt ist und das Attribut „`active`“ des Szenarios den Wert „`False`“ hat, wird es auf „`True`“ gesetzt und der im Szenario gespeicherte Befehl an das Zielgerät gesendet (Zeile 20-21).

Wenn die Bedingung nicht erfüllt ist und das Szenario zuvor aktiv war, so wird das Attribut „`active`“ auf „`False`“ gesetzt (Zeile 22).

Dieser Vorgang stellt sicher, dass die Szenarien nur dann ausgelöst werden, wenn die definierte Bedingung erfüllt ist, und dass Befehle nur dann gesendet werden, wenn ein Szenario ausgelöst wird.

Bereitstellung der Daten

Nach der Verarbeitung und dem Auslösen des Szenarios, werden die Daten über das WebSocket an den Benutzer gesendet.

Dazu wird die Klasse `WebsocketHandler` aus dem Skript `services.py` importiert (Listing 19 Zeile 24). Mit der Funktion `send_data()` dieser Klasse werden die Daten an das aktive WebSocket gesendet (Zeile 25).

Durch diese Architektur kann der Webserver in Echtzeit auf neue MQTT-Nachrichten reagieren und die empfangenen Daten effizient an verbundene Clients weiterleiten.

5.5 WebSocket

Die Geräte senden kontinuierlich Daten über die MQTT-Verbindung an den MQTT-Broker. Diese Daten müssen verarbeitet und dem Benutzer in Echtzeit angezeigt werden. Zu diesem Zweck wird eine WebSocket-Schnittstelle bereitgestellt, die eine direkte und effiziente bidirektionale Kommunikation ermöglicht.

Die Route `mqtt/device/<device_id>` stellt eine WebSocket-Verbindung zur Verfügung, wobei `device_id` für die eindeutige Geräte-ID steht. Die Klasse `WebsocketHandler` in der Datei `services.py` verwaltet sämtliche WebSocket-Verbindungen und deren Lebenszyklus. Das Feldattribut `active_connections` speichert alle aktiven WebSocket-Verbindungen in Form einer „List of Dictionaries“, wobei jedes Dictionary eine bestehende Verbindung mit der zugehörigen `device_id` sowie dem WebSocket-Objekt enthält.

Die Autorisierung des Benutzers, der eine Verbindung zum WebSocket aufbauen möchte, erfolgt über die Funktion `auth_websocket()`. Diese akzeptiert zunächst die WebSocket-Verbindung und wartet asynchron auf die erste Nachricht. Die erste Nachricht, die an dem WebSocket gesendet wird, dient als Initialisierungsnachricht und muss ein Token erhalten, das ähnlich wie ein HTTP-Header aufgebaut ist:

```
"auth": "Bearer <TOKEN>"
```

Aus dem übergebenen Token wird die Benutzer-ID extrahiert. Anschließend wird mit dem Befehl `queries.verify_user_device(...)` geprüft, ob der authentifizierte Benutzer berechtigt ist, auf das angegebene Gerät zuzugreifen. Fehlt das Token oder besitzt der Benutzer keine Zugriffsrechte auf das Gerät, wird eine entsprechende Fehlermeldung ausgegeben und geloggt.

Nach erfolgreicher Autorisierung des Benutzers wird die Funktion `connect()` aus der Klasse `WebsocketHandler` aufgerufen. Diese fügt die neue Verbindung in die Liste von aktiven Verbindungen hinzu. Die Verbindung bleibt bestehen, bis sie entweder vom Benutzer geschlossen oder durch einen Netzwerkfehler unterbrochen wird.

Die Kommunikation über das WebSocket-Protokoll erfolgt in einer Endlosschleife, die ständig auf eingehende Nachrichten wartet. Wird die Verbindung vom Benutzer oder durch andere Ereignisse unterbrochen, wird diese mit der Funktion `disconnect` aus der Liste der aktiven Verbindungen entfernt.

Die Funktion `send_data()` ermöglicht dNas gezielte Senden von Daten über die WebSocket-Schnittstelle. Dabei wird geprüft, ob eine aktive WebSocket-Verbindung mit der entsprechenden `device_id` existiert. Falls ja, werden die zu sendenden Daten als JSON-Objekt über die WebSocket-Schnittstelle an das Gerät übertragen. Tritt ein Übertragungsfehler auf, wird die Verbindung geschlossen und aus der Liste der aktiven Verbindungen entfernt.

Die Implementierung dieser WebSocket-Schnittstelle ermöglicht eine effiziente und latenzarme Datenübertragung zwischen dem MQTT-Broker und den angebundenen Geräten.

5.6 Frontend

TODO Das Frontend der Webanwendung ist als Single Page Application (SPA⁶) implementiert. Eine SPA ist eine Webanwendung, die auf einer einzigen HTML-Seite geladen wird und deren Inhalt dynamisch durch JavaScript aktualisiert wird. Dies führt zu einer verbesserten Benutzererfahrung, da der Wechsel zwischen verschiedenen Ansichten schneller erfolgt und weniger Serveranfragen notwendig sind.

Das Frontend der Anwendung besteht aus einer HTML-Seite zur Strukturierung der Inhalte, JavaScript für die interaktive Funktionalität sowie CSS-Dateien zur Gestaltung und Formatierung. Diese Komponenten sind im Paket `Webserver` organisiert und befinden sich in den Verzeichnissen „`templates`“ und „`static`“.

5.6.1 Dynamische Datendarstellung mit JavaScript

Die Funktionalität des Frontends wird durch ein JavaScript-Skript „`script.js`“ gesteuert.

Die Benutzerdaten werden auf die Seite dynamisch dargestellt. Beim Laden der Seite werden die Benutzerdaten auf dem Server abgefragt und lokal gespeichert. Anschließend werden die Elemente der Oberfläche basierend auf diese Daten dynamisch erzeugt.

Asynchronen Serveranfragen

Da sowohl das Backend als auch die Datenbank asynchron implementiert sind, müssen auch die Anfragen im Frontend asynchron verarbeitet werden. Dies wird durch die Verwendung von asynchronen HTTP-Anfragen mittels der `fetch`-API realisiert. Dadurch wird verhindert, dass die Benutzeroberfläche während der Kommunikation mit dem Server blockiert wird. Antworten des Servers werden mittels Promises verarbeitet, wodurch das Frontend flexibel auf unterschiedliche Antwortzeiten reagieren kann.

⁶Eng. Single Page Applicaiton

Im Listing 20 ist die Funktion `get_user_data` dargestellt. Diese Funktion dient als Beispiel für die asynchrone Verarbeitung von Server-Anfragen im Frontend. Sie nutzt `async` und `await`, um eine GET-Anfrage an die Route „/get_user_data“ zu senden. Dabei wird das Token aus dem „localStorage“ als Bearer-Token im Authorisation-Header der Anfrage übermittelt.

Die Serverantwort wird in einem `try-catch`-Block verarbeitet, um potenzielle Fehler abzufangen und die Anwendung vor unerwarteten Abstürzen zu schützen. Falls die Antwort des Servers nicht den Status `ok` aufweist, wird eine Fehlermeldung generiert. Anschließend wird der zurückgegebene JSON-Inhalt geprüft. Falls ein Fehler im Antwortobjekt enthalten ist, wird dieser ebenfalls als Exception geworfen. Nur wenn die Serverantwort gültig ist und keine Fehlermeldung enthält, wird das empfangene Benutzerobjekt zurückgegeben. Im Fehlerfall wird die Funktion `errorHandler()` aufgerufen, um eine entsprechende Fehlerbehandlung vorzunehmen.

```
async function get_user_data() {
  try {
    const response = await fetch(
      '/get_user_data',
      {
        method: 'GET',
        headers: {
          'auth': `Bearer ${localStorage.getItem('token')}`
        }
      }
    );
    if (!response.ok)
      throw new Error(response.stat);
    else {
      const response_data = await response.json();
      if (response_data.error)
        throw new Error(response_data.error);
      else
        return response_data;
    }
  } catch (error) {
    errorHandler();
  }
}
```

Listing 20: script.js: get_user_data

Benutzerinteraktion Die Reaktionen auf die Benutzerinteraktionen sind mittels Event-Listener implementiert. Event-Listener ermöglichen es, auf Benutzerinteraktionen wie Klicks, Tastendrücke oder Mausbewegungen zu reagieren. In JavaScript werden sie mit der Funktion `addEventListener` an ein HTML-Element gebunden. Dabei wird das gewünschte Ereignis und eine Callback-Funktion angegeben, die beim Auftreten des Ereignisses ausgeführt wird.

Event-Listener können nicht nur für einzelne Elemente, sondern auch für übergeordnete Container genutzt werden. Dies ist besonders nützlich, wenn neue Elemente dynamisch hinzugefügt oder gelöscht werden.

Das Listing 21 zeigt den Prozess der Erstellung eines Formulars zum Hinzufügen eines neuen Hauses. In der ersten Zeile wird ein Event-Listener auf das Element mit der ID „BTN-createHouse“ gesetzt. Beim Anklicken wird die Funktion `mnCreateHouse` aufgerufen, die das Eingabeformular erstellt. Dieser Funktion wird als Parameter ein `div`-Element übergeben, in dem das Formular angezeigt wird. Die HTML-Struktur des Formulars wird in den Zeilen 8 bis 15 definiert.

Anschließend wird ein Event-Listener für die Schaltfläche „BTNcloseHouseMN“ registriert,

der das Formular schließt, indem er den Inhalt des `div`-Elements entfernt und dessen CSS-Klassen zurücksetzt.

Die Schaltfläche „BTNaddHouse“ erhält einen asynchronen Event-Listener, der die Funktion `addHouse` aufruft. Als Parameter wird der Name des Hauses übergeben, den der Benutzer im Eingabefeld „houseName“ eingegeben hat.

```
1 document.getElementById("BTNcreateHouse").addEventListener("click",
2   () => {
3     mnCreateHouse(document.getElementById("MNcreateHouse"));
4   });
5
6 function mnCreateHouse(parent_div){
7   parent_div.innerHTML = `
8     <div class="p-4">
9       <label for="houseName">House Name:</label>
10      <input type="text" id="houseName" value="House 1">
11      <div>
12        <button id="BTNaddHouse">Save</button>
13        <button id="BTNcloseHouseMN">Close</button>
14      </div>
15    </div>
16  `;
17
18  BTNcloseHouseMN.addEventListener("click", () => {
19    parent_div.innerHTML = "";
20    parent_div.classList = "";
21  });
22
23  BTNaddHouse.addEventListener("click", async () => {
24    await addHouse(document.getElementById("houseName").value);
25  });
26 }
```

Listing 21: script.js: mnCreateHouse

Die Implementierung dieser Event-Listener folgt einem einheitlichen Muster, das für weitere Schaltflächen und Formulare in der Anwendung übernommen werden kann. Je nach Anforderung kann die Funktionalität entsprechend angepasst und erweitert werden.

Oberfläche

Die Datei „index.html“ (Listing 22), die sich im Verzeichnis „templates“ befindet, bildet die Grundlage für die Benutzeroberfläche der Anwendung. Sie wird vom Server als Antwort auf einer GET-Anfrage an die Root-Route „/“ bereitgestellt.

Im `head`-Bereich wird zunächst die Zeichenkodierung auf „UTF-8“ gesetzt. Der Meta-Tag „viewport“ gibt an, dass sich die Webseite an die Bildschirmbreite des Gerätes anpasst. Der „<title>“-Tag definiert den Titel der Webseite, der in der Titelleiste des Browsers angezeigt wird. Anschließend wird eine externe CSS-Datei „output.css“ für das Styling der Webseite eingebunden. Mit der Funktion „url_for“ wird die URL zur CSS-Datei dynamisch generiert, so dass ein korrektes Laden sichergestellt ist.

Der `body`-Bereich der HTML-Datei enthält zwei zentrale `div`-Elemente: „navbar“ für die Navigationsleiste und „app“ für den dynamisch generierten Inhalt der Seite. Am Ende der Datei ist ein Verweis auf das JavaScript-Skript „script.js“ eingebunden. Dieses Skript übernimmt die Steuerung des Routings sowie die Interaktion mit dem Benutzer, indem es die Ansicht basierend auf dem Authentifizierungsstatus des Nutzers und geladenen Daten dynamisch aktualisiert.

```
1 <!DOCTYPE html>
```

```

2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Baida Bachelor</title>
7   <link rel="stylesheet" href="{{url_for('static',path='/output.css')}}">
8 </head>
9 <body>
10  <div id="navbar"></div>
11  <div id="app" class="app content"></div>
12  <script src="../static/js/script.js"></script>
13 </body>
14 </html>

```

Listing 22: index.html

Anmeldung und Registrierung

Beim Laden der Seite überprüft die Anwendung, ob ein gültiges Token im `localStorage` vorhanden ist. Falls kein Token existiert oder es ungültig ist, wird der Nutzer automatisch zur Login-Seite weitergeleitet. Andernfalls werden die Benutzerdaten von dem Server abgefragt und die Hauptseite wird geladen.

Die Login-Seite wird in der Funktion „`renderLoginForm()`“ (Listing 23) erstellt. Dabei wird das `div`-Element „`app`“ mit einem Formular versehen, das Eingabefelder für den Benutzernamen und das Passwort enthält (Zeile 1-18).

Die Schaltfläche „Login“ ist mit einem Event-Listener für das Ereignis „submit“ verknüpft. Durch Drücken des Login-Buttons werden die Daten aus den Eingabefeldern abgelesen und in den Variablen „`username`“ und „`password`“ gespeichert (Zeile 22-23). Anschließend wird eine asynchrone Post-Anfrage an die Route „`/login`“. Im `header`-Objekt wird der `Content-Type` auf „`application/json`“ gesetzt, damit der Server weiß, dass die übermittelten Daten im JSON-Format vorliegen (Zeile 30). Die Anmeldedaten werden mit `JSON.stringify()` in eine JSON-Zeichenkette umgewandelt und im `body` der Anfrage übermittelt (Zeile 32).

Nach dem Senden der Anfrage wartet die Anwendung mit `await` auf die Antwort des Servers. Sobald die Antwort eingetroffen ist, wird sie mit der Funktion `response.json()` in ein JSON-Objekt umgewandelt und in der Variablen `response_data` gespeichert (Zeile 35).

Stimmen die übermittelten Anmeldedaten mit den hinterlegten Benutzerinformationen überein, antwortet der Server mit einem JWT-Token. Dieses Token wird im `localStorage` gespeichert, wodurch der Benutzer authentifiziert und angemeldet wird (Zeile 37). Andernfalls antwortet der Server mit einem Fehler, dass die falschen Daten eingegeben wurden und die entsprechende Meldung wird ausgegeben.

```

1  app.innerHTML = `
2    <div class="flex items-center justify-center h-screen">
3      <h2>LOGIN</h2>
4      <form id="loginForm" class="space-y-4">
5        <div>
6          <label for="username">Username</label>
7          <input type="text" id="username">
8        </div>
9        <div>
10         <label for="password">Password</label>
11         <input type="password" id="password">
12       </div>
13       <button type="submit">LOGIN</button>
14     </form>
15   </div>

```

```

16         <button id="BTNsignup">SignUp here</button>
17     </div>
18 </div>`;
19
20     document.getElementById('loginForm').addEventListener('submit',
21         async (event) => {
22         event.preventDefault();
23         const username = document.getElementById('username').value;
24         const password = document.getElementById('password').value;
25
26         // asynchrone Anfrage auf die Route "login"
27         try {
28             const response = await fetch('/login', {
29                 method: 'POST',
30                 headers: {
31                     'Content-Type': 'application/json'
32                 },
33                 body: JSON.stringify({ username, password })
34             });
35             //
36             const response_data = await response.json();
37             if ('token' in response_data) {
38                 localStorage.setItem('token', response_data['token']);
39                 appRouter();
40             }
41         } catch(error){ errorHandler(error);}

```

Listing 23: script.js: renderLoginForm

Unter dem Login-Button befindet sich ein Link zur Registrierungsseite. Ein Klick auf diesen Link ruft die Funktion `renderSignUpPage` auf, die das `div`-Element „app“ mit einem Formular für die Benutzerregistrierung füllt. Das Formular enthält Eingabefelder für den Benutzernamen, die E-Mail-Adresse und das Passwort. Durch Drücken der Taste „Sign Up“ werden die Anmeldedaten an die Route „/sign_up“ mittels einer Post-Anfrage gesendet. Wenn die Registrierungsdaten gültig sind, antwortet der Server mit einem JWT-Token und der Benutzer wird im System registriert.

Navigationsleiste

Die Funktion `renderNavbar` füllt das `div`-Element „navbar“ mit einer Navigationsleiste. Diese enthält einen Link zur Hauptseite sowie die Schaltflächen „My Devices“, „My Scenarios“ und „Logout“.

Ein Klick auf „Logout“ löscht das gespeicherte Token und leitet den Benutzer zur Anmeldeseite weiter. Die Schaltfläche „My Devices“ ruft die Funktion `renderMyDevicePage` auf, die die gespeicherten Geräte des Benutzers anzeigt. Mit „My Scenarios“ wird die Funktion `renderMyScenariosPage` aufgerufen, die die vom Benutzer erstellten Szenarien darstellt.

Hauptseite

Nach erfolgreicher Anmeldung und Erhalt des JWT-Tokens wird der Benutzer zur Hauptseite der Anwendung weitergeleitet. Dort erfolgt eine Anfrage an die Route „/get_user_data“, um die relevanten Benutzerdaten vom Server abzurufen. Die Serverantwort auf diese Anfrage besteht aus einem JSON-Objekt, das Listen der gespeicherten Häuser, Geräte und Szenarien enthält. Diese Daten werden anschließend in der lokalen Variablen „stored_data“ gespeichert und zur dynamischen Darstellung der Benutzeroberfläche verwendet. Falls die Token-Validierung fehlschlägt, wird der Benutzer automatisch zur Anmeldeseite zurückgeleitet. Nach Erhalt der Benutzerdaten wird die Hauptseite generiert.

Die Funktion `renderMainPage()` füllt das `div`-Element „app“ mit der im Listing 24 dargestellten HTML-Struktur.

```
<div id="main_page">
  <div id="main_header">
    <p>Hello, ${data.user_data.username}!</p>
    <button id="BTNcreateHouse" type="button">Create New House</button>
  </div>
  <div id="MNcreateHouse"></div>
  <div id="houseList"></div>
</div>
```

Listing 24: html: renderMainPage

Oben auf der Seite wird eine Willkommensnachricht zusammen mit einer Schaltfläche zum Erstellen eines neuen Hauses angezeigt. Durch Anklicken des Buttons wird das `div`-Element „MNcreateHouse“ mit einem Formular befüllt, in das der Benutzer den Namen des Hauses eingeben kann. Mit der Schaltfläche „Save“ wird das Haus gespeichert. Dabei wird eine asynchrone Anfrage an die Route „add_house“ gesendet, die das Token und den Hausnamen enthält, und das Haus wird in der Datenbank gespeichert. Nach Abschluss wird die Seite neu geladen und das neue Haus erscheint.

In das `div`-Element „houseList“ werden die vom Benutzer erstellten Häuser, einschließlich der Räume und zugehörigen Geräte, geladen. Diese Daten sind in der lokalen Variablen „stored_data“ unter dem Schlüssel „houses“ gespeichert. Für jedes Haus wird ein `div`-Element mit der im Listing 25 dargestellten HTML-Struktur erstellt und an das `div`-Element „houseList“ angehängt. Dabei wird jede ID um die ID des jeweiligen Hauses erweitert, um die spätere Zuordnung der Räume und Geräte zu ermöglichen.

```
<div id="header${house.id}">${house.name} </div>
<div id="house_element${house.id}">
  <div>
    <button id="BTNcreateRoom${house.id}">New Room</button>
    <div id="create_room${house.id}"></div>
    <div id="room_list${house.id}"></div>
  </div>
  <button id="BTNdeleteHouse${house.id}">Delete House</button>
</div>
</div>
```

Listing 25: html: house_element

Im `header`-Element wird der Name des Hauses angezeigt. Unterhalb des Headers befindet sich eine Schaltfläche zum Anlegen eines neuen Raumes sowie der Bereich für die Auflistung der Räume. Zusätzlich gibt es eine Schaltfläche zum Löschen des Hauses. Der Button „create_room“ öffnet ein Formular, in dem der Benutzer den Namen des Raumes eingeben und speichern kann.

Anschließend wird das `div`-Element „room_list“ mit den Räumen des Hauses gefüllt. Dieser Vorgang ist ähnlich wie bei der Anzeige der Häuser. Für jeden Raum wird ein `div`-Element „room_element“ mit der ID des Raumes erzeugt. Dieses Element enthält den Namen des Raumes, eine Schaltfläche zum Hinzufügen eines Gerätes, den Bereich zur Auflistung der Geräte und eine Schaltfläche zum Löschen des Raumes.

Die Schaltfläche zum Hinzufügen eines Geräts öffnet ein Formular, in das der Benutzer die ID und den Namen des Gerätes eingeben soll. Durch Bestätigen der Schaltfläche „Speichern“ wird das Gerät gespeichert und dem Raum zugeordnet.

Die zu dem Raum zugeordneten Geräte müssen auch angezeigt werden. Für jedes Gerät wird ein `div`-Element „device_element“ mit der Geräte-ID erzeugt, wie in der Abbildung 3 gezeigt.

Eine detaillierte Beschreibung des Elements folgt weiter unten.

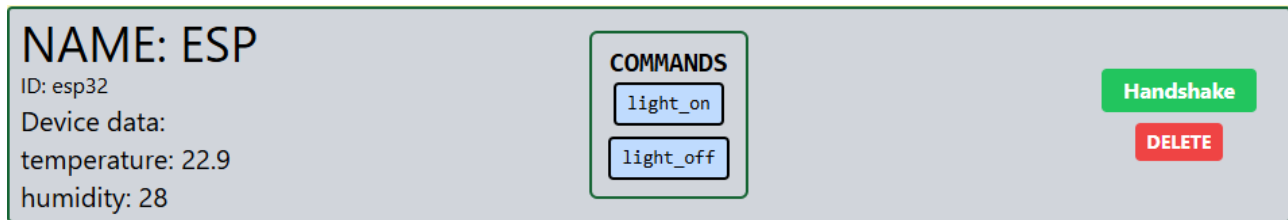


Abbildung 3: device_element

My Devices

Durch das Anklicken an die Schaltfläche „My Devices“ an der Navigationsleiste, wird die Funktion `renderMyDevicesPage()` aufgerufen, die die vom Benutzer angemeldeten Geräte darstellt.

Für jedes gespeicherten Gerät wird ein `div`-Element „device_element“ mit der im Listing 26 dargestellten HTML-Struktur erstellt. Das `div`-Element ist auf der Abbildung 3 dargestellt.

Die vom Gerät empfangenen Daten werden im `div`-Element „deviceData“ dargestellt. Über eine WebSocket-Verbindung werden die Daten in Echtzeit empfangen, angezeigt und kontinuierlich aktualisiert. Eine detaillierte Beschreibung folgt im Paragraph „WebSocket“ dieses Abschnitts.

```
<div id="device_element${device.dev_id}">
  <p>NAME: ${device.name}</p>
  <h1>DEV_ID: ${device.dev_id}</h1>
  <p>Description: ${device.description ? device.description : ""}</p>
  <p>ROOM: ${device.room ? device.room : "-"}</p>
  <p>Data fields: ${device.data_fields}</p>
  <div id="deviceData${device.dev_id}"></div>
</div>
<div id="commands${device.dev_id}"></div>
<div>
  <button id="BTNhandshake${device.dev_id}">Handshake</button>
  <button id="BTNdeleteDevice${device.dev_id}">Delete</button>
</div>
```

Listing 26: html: device_element

My Scenarios

Durch das Anklicken an die Schaltfläche „My Scenarios“ an der Navigationsleiste, wird die Funktion `renderMyScenariosPage()` aufgerufen, die die vom Benutzer erstellten Szenarien darstellt.

Das `div`-Element „app“ wird mit der im Listing 27 dargestellten HTML-Struktur befüllt.

```
<div class="main_page">
  <div class="main_header">
    <button id="BTNcreateScenario" type="button" class="
      create_house-btn">Create Scenario</button>
  </div>
  <div id="MNcreateScenario"></div>
  <div id="scenarioList" class="device_list"></div>
</div>
```

Listing 27: html: renderMyScenariosPage

Ein Klick auf die Schaltfläche mit der ID „BTNcreateScenario“ füllt das div-Element „MNcreateScenario“ mit dem auf die Abbildung 4 dargestellten Formular zur Erstellung eines neuen Szenarios. Das Formular enthält die Felder zur Eingabe der nötigen Szenariodaten.

Das Szenario kann ein beliebiger Name haben. In den Auswahllisten „Source Device“ und „Target Device“ werden die Namen der bereits registrierten Geräte angezeigt. Nach dem Auswahl des Quellgeräts werden die zugehörigen Datenfelder dynamisch in der Auswahlliste „Data Field“ geladen.

Als Bedingung („Condition“) kann der Benutzer zwischen den Vergleichsoperatoren „<“, „>“ und „=“ wählen. Im Feld „Value“ wird eine Zahl eingegeben, der als Vergleichswert in der Bedingung dient.

Nach Auswahl des Zielgeräts werden in die Liste „Command“ die gespeicherten Befehle geladen, die das Zielgerät ausführen kann.

Dabei werden Quell- und Zielgerät sowie ihre Datenfelder und Befehle dynamisch aus der lokalen Variable `stored_data` geladen.

Durch das Anklicken auf die Schaltfläche „BTNaddScenario“ des Formulars wird ein Post-Request auf die Route „add_scenario“ mit der Szenariodaten im Body gesendet. Damit wird das Szenario in der Datenbank gespeichert.

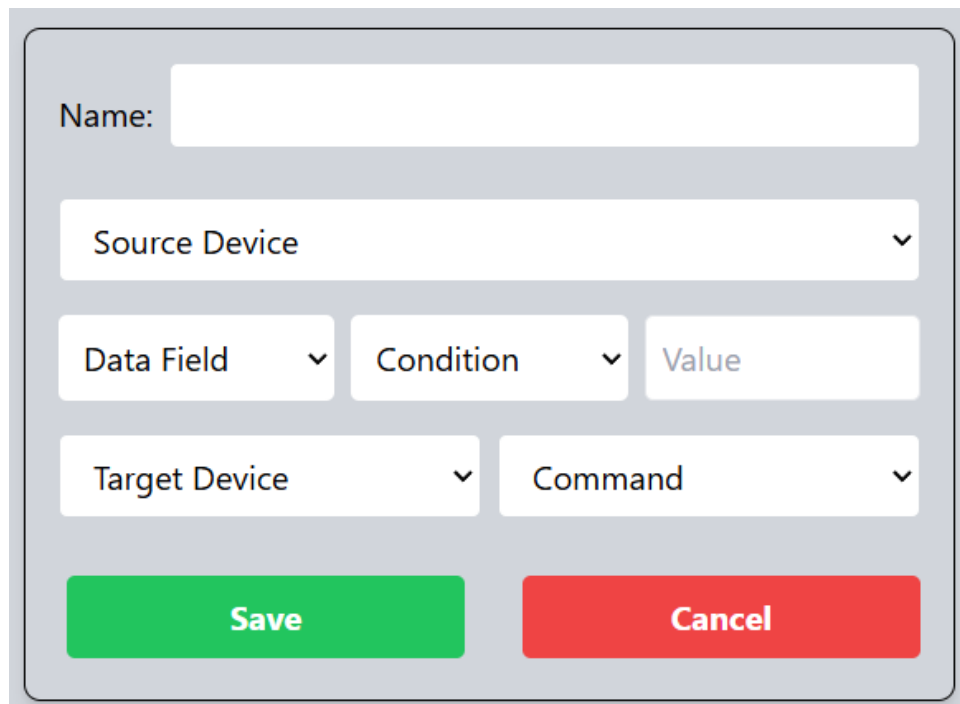
Das Formular ist in einem grauen Rahmen mit abgerundeten Ecken und einer hellgrauen Hintergrundfarbe eingebettet. Es besteht aus folgenden Elementen: Ein Textfeld mit der Beschriftung 'Name:' und einem weißen Eingabefeld. Darunter befindet sich eine Dropdown-Liste mit der Beschriftung 'Source Device' und einem nach unten gerichteten Pfeil. Darunter befinden sich drei separate Eingabefelder: 'Data Field' mit einer Dropdown-Pfeil-Symbol, 'Condition' mit einer Dropdown-Pfeil-Symbol und 'Value' als reines Textfeld. Darunter befinden sich zwei weitere Dropdown-Liste: 'Target Device' und 'Command', jeweils mit einem nach unten gerichteten Pfeil. Am unteren Rand befinden sich zwei große, rechteckige Buttons: ein grüner Button mit der Aufschrift 'Save' und ein roter Button mit der Aufschrift 'Cancel'.

Abbildung 4: Formular zur Erstellung eines Szenarios

Das div-Element „scenarioList“ wird mit den von Benutzer erstellten Szenarien befüllt. Für jedes Szenario wird ein div-Element „scenario_element“ mit der ID des Szenarios erstellt.

Die Abbildung 5 stellt ein „scenario_element“ dar. Das dargestellte Szenario hat den Namen „Abend“. Wenn das Datenfeld „light“ des Gerätes mit der ID „m5bmevcn1“ den Wert unter 20 liefert, so wird das Befehl „light_turn_on“ an das Gerät mit der ID „esp32light“ gesendet.

Durch das Anklicken auf die Schaltfläche „Delete“ wird ein Delete-Request auf die Route „delete_scenario“ mit dem Primärschlüssel des Szenarios gesendet und das Szenario wird damit aus der Datenbank gelöscht.

WebSocket

NAME: Abend
IF: m5bmevcnl.light < 20
THEN: esp32light => light_turn_on

Delete

Abbildung 5: scenario_element

Im `div`-Element „deviceData“ werden die vom Gerät empfangenen Daten angezeigt. Dazu wird eine WebSocket-Verbindung aufgebaut. Das Listing 28 zeigt die Initialisierung der Verbindung sowie die Verarbeitung der eingehenden Daten.

Zunächst wird ein WebSocket-Objekt mit der URL der WebSocket-Route erzeugt (Zeile 1-3). Sobald die Verbindung erfolgreich hergestellt wurde, sendet das Skript das Authentifizierungstoken an den Server (Zeile 5-8), um die Verbindung zu autorisieren.

Bei jeder eingehenden Nachricht wird das Ereignis `onmessage` ausgelöst (Zeile 10). Die empfangenen Daten werden in ein JavaScript-Objekt umgewandelt (Zeile 11) und anschließend wird das `div`-Element mit der ID „deviceData“ für das entsprechende Gerät abgerufen (Zeile 12).

Wenn das Gerät keine vordefinierten Datenfelder hat, werden die Schlüssel des empfangenen Datenobjekts extrahiert und als Datenfelder verwendet (Zeilen 13-16). In diesem Fall werden alle empfangenen Daten angezeigt.

Für jedes dieser Datenfelder wird geprüft, ob es im empfangenen Datenobjekt vorhanden ist (Zeile 17-18). Ist dies der Fall, sucht das Skript nach einem `p`-Element, dessen ID den Namen des Datenfelds und die Geräte-ID enthält (Zeile 19-20). Ist ein solches Element nicht vorhanden, wird es dynamisch erzeugt und in das `div`-Element eingefügt (Zeile 21-25). Schließlich wird der Inhalt des Elements mit dem Schlüssel und dem entsprechenden Wert aus den empfangenen Daten aktualisiert (Zeile 26).

Wenn die Verbindung unterbrochen wird, wird eine Meldung an die Konsole ausgegeben.

```
1  const ws = new WebSocket(  
2    `ws://127.0.0.1:8000/mqtt/device/${device.dev_id}`  
3  );  
4  
5  ws.onopen = () => {  
6    ws.send(JSON.stringify({ auth: "Bearer " +  
7      localStorage.getItem("token") }));  
8  };  
9  
10 ws.onmessage = (event) => {  
11   const data = JSON.parse(event.data);  
12   const parent_div = device_element.querySelector(`#deviceData${  
13     device.dev_id}`);  
14   let data_fields = device.data_fields;  
15   if (data_fields.length == 0) {  
16     data_fields = Object.keys(data);  
17   }  
18   for (const key of data_fields) {  
19     if (data.hasOwnProperty(key)) {  
20       let dataField = device_element.querySelector(  
21         `#${key}${device.dev_id}`);  
22       if (!dataField) {  
23         dataField = document.createElement('p');  
24         dataField.id = `#${key}${device.dev_id}`;  
25         parent_div.appendChild(dataField);  
26       }  
27       dataField.innerHTML = `${key}: ${data[key]}`  
28     }  
29   }  
30 }
```

```

28     }
29 };
30
31 ws.onclose = () => {
32     console.error(`WebSocket for device ${device.dev_id} closed`);
33 };

```

Listing 28: script.js: websocket

5.6.2 Styling

Das Styling der Webseite ist mit **Tailwind CSS** definiert. **Tailwind CSS** hat den Vorteil vor klassischem CSS, dass es vordefinierte Klassen hat, die direkt im HTML-Code angebunden werden können. Außerdem ist es möglich eine separate **.css**-Datei zu erstellen, wo weiteren Klassen definiert werden können.

Einrichtung von Tailwind CSS

Tailwind CSS wird über **npm** installiert. Zunächst musste Node.js auf dem System vorhanden sein. Danach wurde **Tailwind CSS** mit folgendem Befehl installiert:

```
npm install -D tailwindcss postcss autoprefixer
```

Nach der erfolgreichen Installation muss die Konfigurationsdatei erstellt werden. Mit diesem Befehl wird die Konfigurationsdatei „**tailwind.config.js**“ erstellt:

```
npx tailwindcss init -p
```

Aufbau mit Tailwind CSS

Für das Styling der Webseite wurde eine eigene Datei **styles.css** (Listing 29) erstellt. Damit die vordefinierten Klassen von **Tailwind CSS** verwendet werden können, müssen am Anfang der Datei die **Tailwind CSS**-Direktiven eingebunden werden (Zeile 1–3). Diese Direktiven stellen sicher, dass die Basisstile, Komponentenklassen und Utility-Klassen von **Tailwind** korrekt geladen und verarbeitet werden. In der Datei können eigene CSS-Klassen definiert werden, die mit dem Befehl **@apply** mehrere **Tailwind CSS**-Klassen zusammenfassen.

Ein Beispiel ist die Klasse **main_page**, die für den Hauptbereich der Webseite verwendet wird. Diese Klasse enthält mehrere **Tailwind CSS**-Regeln:

- **p-1** setzt den Innenabstand (Padding) auf 4 Pixel. Damit wird sichergestellt, dass Inhalte nicht direkt an den Rändern des Elements anliegen.
- **my-5** legt den vertikalen Außenabstand (Margin) auf 20 Pixel fest, um das Element optisch von anderen Elementen abzugrenzen.
- **h-full** stellt sicher, dass das Element die gesamte verfügbare Höhe seines übergeordneten Containers einnimmt.
- **bg-gray-300** definiert einen hellgrauen Hintergrund.
- **items-center** richtet die enthaltenen Elemente mittig aus.

```
1  @tailwind base;
2  @tailwind components;
3  @tailwind utilities;
4
5  .main_page {
6      @apply p-1 my-5 h-full bg-gray-300 items-center;
7  }
8  ...
```

Listing 29: style.css

Damit die Klassen übernommen und kompiliert werden können, muss der Watch-Modus von Tailwind CSS gestartet werden:

```
npx tailwindcss -i ./app/webserver/static/styles.css \
-o ./app/webserver/static/output.css --watch
```

Der Parameter `-i` gibt den Pfad zu der Eingabedatei an, in der die eigenen Tailwind CSS-Klassen definiert sind. Der Parameter `-o` legt die Ausgabedatei fest, in die der kompilierte CSS-Code geschrieben wird. Durch `--watch` wird der Watch-Modus aktiviert, wodurch Änderungen in der Eingabedatei automatisch erkannt und in die Ausgabedatei übernommen werden. Dadurch wird sichergestellt, dass neue oder geänderte Tailwind CSS-Klassen direkt im Projekt verfügbar sind, ohne dass der Compiler manuell neu gestartet werden muss.

6 Ergebnisse und Diskussion

Das in dieser Bachelorarbeit entwickelte System dient der Steuerung und Überwachung von Umgebungsbedingungen im Haus. Für diese Aufgaben wurden verschiedene Komponenten und Technologien eingesetzt.

Im Zentrum des Systems liegt ein Raspberry Pi, der als WLAN-Access-Point konfiguriert wurde. Gleichzeitig fungiert er als MQTT-Broker bereitstellt und hostet einen Webserver. Dadurch funktioniert das System vollständig lokal und ist unabhängig von der Internetverbindung.

Die Kommunikation zwischen den Komponenten erfolgt über das MQTT-Protokoll. Die Geräte verbinden sich mit dem WLAN-Access-Point und senden ihre Daten in eigenen Topics an den MQTT-Broker. Zudem können sie die Steuerbefehle empfangen, die über das Webinterface von Benutzer gesendet werden.

Zur Verwaltung der Geräte und zur Darstellung der empfangenen Daten wurde eine Webschnittstelle entwickelt. Diese Schnittstelle bietet eine intuitive Bedienung des Systems. Die Benutzer können sich registrieren, eigene Häuser mit Räumen anlegen und die Geräte hinzufügen. Die von Geräten gesendeten Daten werden auf der Weboberfläche dargestellt. Darüber hinaus haben die Benutzer die Möglichkeit eigene Szenarien zu erstellen, die bei Erfüllung eingetragener Bedingungen automatisch ausgeführt werden.

Die modulare Architektur des Systems erlaubt es, verschiedene Mikrocontroller und Sensoren flexibel in das System zu integrieren. Prinzipiell kann jedes Gerät, das sich mit dem WLAN verbindet und MQTT unterstützt, in das System integriert werden.

Im Folgenden wird ein Anwendungsfall beschrieben, der exemplarisch zeigt, wie das entwickelte System in der Praxis eingesetzt werden kann.

6.1 Anwendung des Systems

Vorbereitung der Komponenten

Vor der Verwendung des Systems ist die Einrichtung aller Komponenten erforderlich.

Zunächst muss der Raspberry als WLAN-Access-Point und MQTT-Broker eingerichtet werden. Dafür wird der Raspberry Pi über das Netzkabel mit dem Internet verbunden und die erforderliche Linux-Pakete werden geladen. Die Software `hostapd` [13] richtet den WLAN-Access-Point auf dem Raspberry Pi ein, während `dnsmasq` [14] das lokale Netzwerk konfiguriert. Der MQTT-Broker `mosquitto` [15] wird ebenfalls geladen und gestartet.

Anschließend wird die Webanwendung auf dem Raspberry Pi installiert. Dazu ist Python in Version 3.9 erforderlich. Der Quellcode der Anwendung befindet sich im Projektordner „code/webapp“ und muss auf den Raspberry Pi übertragen werden. Die benötigten Python-Bibliotheken sind in der Datei `requirements.txt` gelistet und müssen installiert werden. Die Anwendung wird mit dem Befehl „`python3 -m app`“ gestartet.

Der Raspberry Pi ist nun vollständig eingerichtet und wird von dem Internet getrennt.

Als Nächstes werden die Hardwarekomponenten eingerichtet.

Die Anbindung der Komponenten an den Arduino ist in der Projektarbeit 1 [1] ausführlich beschrieben. Der Programmcode für den Arduino befindet sich im Projektordner „code/arduino“ und muss auf den Mikrocontroller übertragen werden.

An den ESP-Modulen müssen die erforderlichen Sensoren und Aktoren entsprechend der Beschreibung in Kapitel 4.3 angebunden werden. Der zugehörige Programmcode befindet sich im Projektordner „code/esp“ in jeweils einem Unterordner pro Modul und muss auf das entsprechende Modul übertragen werden.

Damit ist das gesamte System komplett eingerichtet und betriebsbereit.

Verwendung des Systems

Nach der Einrichtung aller Komponenten, müssen sich die WLAN-fähigen Module mit dem von Raspberry Pi bereitgestellten Netzwerk verbinden. Das Netzwerk hat folgende Zugangsdaten:

SSID:	RaspEsp
Password:	mqtt1234

Wie auf an der Seite 24 beschrieben, müssen die Daten des Netzwerks an die Geräten übergeben werden. Geräte mit integriertem Display zeigen einen entsprechenden Hinweis zur Verbindung an.

Sobald die Geräte mit dem Netzwerk verbunden sind, beginnen sie ihre Daten zu senden. Die Steuerung der Geräte und die Überwachung der Daten ist jedoch erst möglich, wenn sowohl der Benutzer als auch die Geräte im System registriert sind.

Der Benutzer verbindet sich mit seinem eigenem Smartphone oder Laptop mit dem vom Raspberry Pi bereitgestellten WLAN und ruft im Webbrowser die Adresse `127.0.0.1:8000` auf. Es wird dem Benutzer angeboten ein Benutzerkonto anzulegen oder sich mit bereits erstellten Konto anzumelden.

Für die Registrierung gibt der Benutzer einen beliebigen Benutzernamen, seine Email-Adresse und ein beliebiges Passwort ein. Damit ist das Konto erstellt und der Benutzer wird eingeloggt und zur Hauptseite weitergeleitet.

Auf der Hauptseite kann der Benutzer ein neues Haus anlegen. Durch das Anklicken auf die Schaltfläche „**Create New House**“ öffnet sich ein Formular, in das der Benutzer einen beliebigen Namen für das Haus eingeben kann. Der Benutzer kann mehrere Häuser anlegen.

Wenn der Benutzer auf den Haus-Block klickt, erscheint eine Schaltfläche, mit der ein Raum im Haus erstellt werden kann. Der Benutzer kann ein Raum mit eindeutigen Namen hinzufügen. In einem Haus können mehrere Räume erstellt werden.

Im Raum-Block gibt es eine Schaltfläche, um ein Gerät anzumelden. Ein Klick auf diese Taste öffnet ein Formular zur Geräteanmeldung, das in der Abbildung 6 dargestellt ist.

▽ Home

New Room

▽ Wohnzimmer

Add New Device

ID:

Name: **Same to ID**

Save **Cancel**

NO DEVICES IN THIS ROOM

Delete Room

Delete House

Abbildung 6: Formular zur Anmeldung eines Geräts im Raum

Der Benutzer gibt die Geräte-ID und einen eindeutigen Namen ein und speichert diese. Damit ist das Gerät registriert und dem Raum zugeordnet.

In diesem Beispiel wird ein ESP-Modul mit dem Namen „ESP32“ und mit der Geräte-ID „esp32“ registriert. Die Abbildung 7 zeigt das angemeldete Gerät in dem Raum-Block.

Links im Geräte-Block sind die Gerätedaten angezeigt. In der Mitte sind die Befehle gelistet, die das Gerät ausführen kann. Das Befehl wird beim Klick sofort an das Gerät gesendet. Rechts gibt es eine Schaltfläche zum manuellen Senden der Handshake-Nachricht sowie zum Löschen des Geräts.

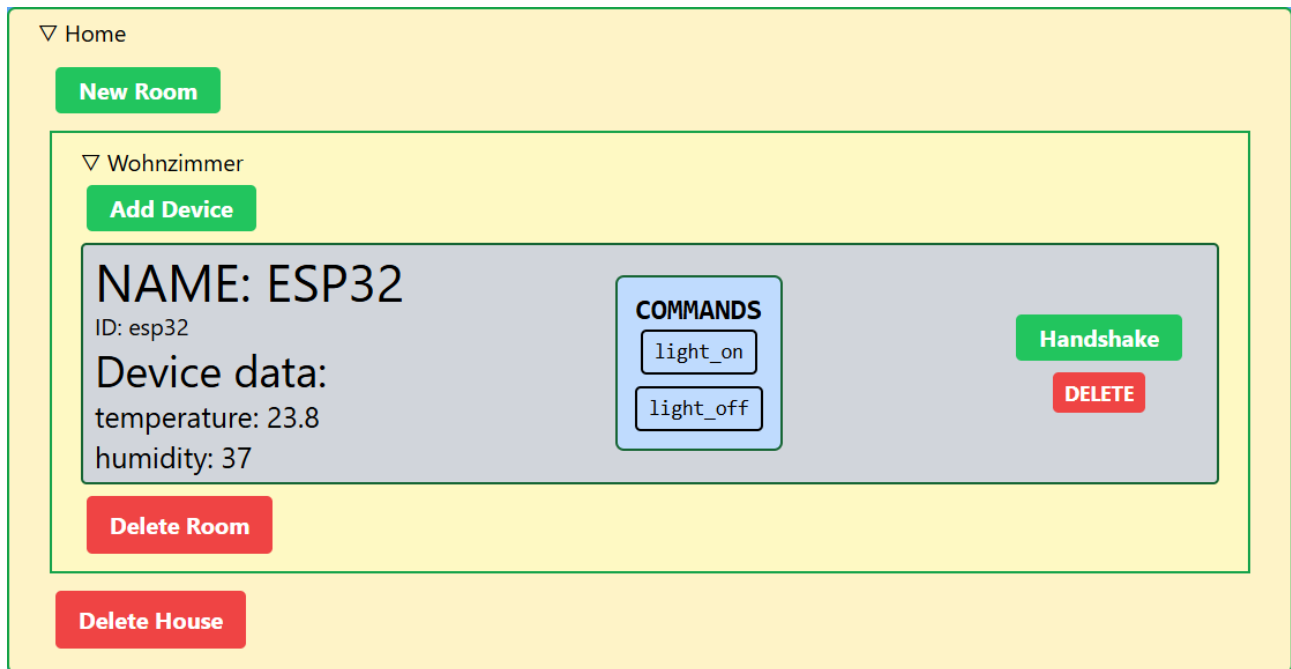


Abbildung 7: Gerät in dem Raum

Mit dem gleichen Verfahren kann der Benutzer weitere Geräte hinzufügen. Für die Registrierung ist es nicht erforderlich, dass das Gerät eingeschaltet oder mit dem Raspberry Pi verbunden ist.

Die Registrierung erfolgt unabhängig davon, ob bereits Daten vom Gerät empfangen wurden. Sobald das Gerät seine Daten an den MQTT-Broker sendet, werden diese unmittelbar auf der Benutzeroberfläche angezeigt. In diesem Fall kann der Benutzer über die Schaltfläche „Handshake“ manuell eine Handshake-Nachricht an das Gerät zu senden. Dadurch werden die Konfigurationsdaten wie verfügbare Datenfelder und unterstützte Befehle abgerufen und aktualisiert.

Nun sind die Geräte angemeldet und der Benutzer kann sowohl ihre Daten sehen, als auch die Steuerbefehle an die Geräte senden.

Zusätzlich hat der Benutzer die Möglichkeit eigene Automatisierungsszenarien zu erstellen. Ein Klick auf die Schaltfläche „My Scenarios“ öffnet die Seite mit den erstellten Szenarien. Oben auf der Seite liegt die Taste „Create Scenario“, die das in Abbildung 4 dargestellte Formular zum Erstellen des Szenarios öffnet.

In diesem Beispiel wird ein Szenario zum Einschalten der Beleuchtung erstellt. Das Ziel ist es, die Beleuchtung einzuschalten, wenn das Datenfeld „light“ einen Wert kleiner als 20 aufweist.

Das Szenario wurde „Abend“ genannt. Als Quellgerät wurde das Modul „M5StickC“ ausgewählt, das Helligkeitswerte liefert. Als Datenfeld wurde „light“ und als „Condition“ das Symbol „<“ ausgewählt. Im Feld „Value“ wurde der Wert 20 eingegeben. Als Zielgerät wurde das ESP32 gewählt, und als „Command“ wurde „light_on“ ausgewählt, um die Beleuchtung

einzuschalten. Die Abbildung stellt das ausgefüllte Formular dar.

In diesem Beispiel wäre es sinnvoll, ein zusätzliches Szenario zum Ausschalten des Lichts zu erstellen. Als „**Condition**“ wird hierfür das Symbol „>“ gewählt und im Feld „**Value**“ der Wert 30 eingetragen. Das Zielgerät ESP32 soll in diesem Fall den Befehl „**light_off**“ ausführen.

Die beiden Szenarien werden wie auf der Abbildung 111 an der Benutzeroberfläche dargestellt.

Nach dem Speichern wird das Szenario automatisch ausgeführt. Der Benutzer kann es jederzeit über die Schaltfläche „**Delete**“ löschen.

Damit wird die Beleuchtung automatisch eingeschaltet, wenn es dunkel wird, und automatisch ausgeschaltet, wenn es hell genug ist.

Mit diesem Verfahren wurde gezeigt, dass das System wie geplant funktioniert. Das System zeigt sich in der Praxis als zuverlässig und stabil. Alle Komponenten konnten erfolgreich eingerichtet und reibungslos in das System integriert werden. Die Geräte lassen sich ohne Probleme registrieren und steuern. Die Interaktion zwischen Benutzer und seine Geräte erfolgt schnell und stabil. Die erstellte Szenarien werden automatisch und ohne Konflikte ausgelöst. Damit erfüllt das System alle gestellten Anforderungen und ist einsatzbereit.

6.2 Vorschläge zur Weiterentwicklung und Erweiterung

Das in dieser Bachelorarbeit entwickelte System stellt eine Grundlage für ein großes Smart-Home-System dar. Das System eignet sich besonders für die Häuser mit weniger als 10 gleichzeitig verwendeten Geräten. Für die Anwendung im realen Betrieb muss das System noch weiterentwickelt werden.

Derzeit dient der Raspberry Pi als zentraler Server des Systems und das System ist ausschließlich über das lokale Netzwerk erreichbar. Für die Weiterentwicklung ist es sinnvoll das System über das Internet erreichbar zu machen. Dadurch könnten die Benutzer ihre Geräte aus der Ferne steuern und überwachen. Zu diesem Zweck müssten der Webserver sowie der MQTT-Broker auf einen externen Server mit fester IP-Adresse und Domainnamen ausgelagert werden. Die Geräte würden sich dann mit dem Internetrouter im Haus verbinden und über das Internet mit dem MQTT-Broker kommunizieren.

Dabei sind verschiedene Sicherheitsaspekte zu berücksichtigen. Das System muss zuverlässig gegen externe Angriffe geschützt werden.

Einer dieser Aspekte betrifft die Anmeldung neuer Geräte im System. Derzeit kann jeder, der die Geräte-ID eingibt, das Gerät anmelden und steuern. Angreifer könnten durch sogenanntes „Brute-Forcing“ eine gültige Geräte-ID finden und somit die Kontrolle über ein Gerät übernehmen.

Um das zu verhindern, könnte eine physische Taste am Gerät angebracht werden, die das Gerät in den Anmeldungsmodus umschaltet. Nach dem Drücken dieser Taste hätte der Benutzer ein begrenztes Zeitfenster, um das Gerät im System anzumelden. Dadurch muss der Benutzer physischen Zugriff auf das Gerät haben, um es anmelden zu können.

7 Quellen

Literatur

- [1] O. Baida, *Anbindung der Sensoren und Aktoren an den Arduino zur Realisierung eines Sicherheitssystems*, Projektarbeit 1, 2024.
- [2] O. Baida, Projektarbeit 2 *Sicherheitssystem für das Haus basierend auf Arduino, ESP8266 & Raspberry Pi* <https://github.com/oleksiibaida/PA2.git>
- [3] Statista, „Smart Home - Anzahl der Haushalte in Deutschland 2028“, Verfügbar unter <https://de.statista.com/prognosen/885611/anzahl-der-smart-home-haushalte-in-deutschland> Zugriff: 13. Januar 2025. [Online].
- [4] Udo Brandes: *Mikrocontroller ESP32: Das Umfassende Handbuch*. Bonn: Rheinwerk Verlag, 2024, ISBN 9783367102860.

Links zur verwendeten Hardware:

- [5] Arduino.cc, *Arduino UNO*, <https://docs.arduino.cc/hardware/uno-rev3/>
- [6] Raspberry Pi Foundation, *Raspberry Pi 1 B+*, <https://www.raspberrypi.com/products/raspberry-pi-1-model-b-plus/>
- [7] Espressif, *ESP8266*, <https://www.espressif.com/>, <https://www.electronicwings.com/sensors-modules/esp8266-wifi-module>
- [8] Bosh, *BME680 - Datasheet*, <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme680-ds001.pdf>, Zugriff: 4. Februar 2025.
- [9] Vishay, *VCNL4040 - Datasheet*, <https://www.vishay.com/docs/84274/vcnl4040.pdf>, Zugriff: 5. Februar 2025.
- [10] Adafruit, *OLED FeatherWing*, <https://learn.adafruit.com/adafruit-128x64-oled-featherwing/downloads>, Zugriff: 16. Januar 2025 [Online].

Links zur verwendeten Software:

- [11] Dr Andy Stanford-Clark, Arlen Nipper, *Message Queuing Telemetry Transport*, <https://mqtt.org/> Zugriff am 15. Februar 2025 [Online].
- [12] NXP Semiconductors, *I2C-bus specification and user manual*, <https://www.nxp.com/docs/en/user-guide/UM10204.pdf> Zugriff am 15. Februar 2025 [Online].

Linux-Paketete:

- [13] Jouni Malinen, *hostapd*, <https://w1.fi/hostapd/>, Zugriff am: 19. September 2024.
- [14] Simon Kelley, *dnsmasq*, <https://dnsmasq.org/doc.html>, Zugriff am: 20. September 2024.
- [15] Eclipse Foundation, *Eclipse Mosquitto*, <https://mosquitto.org/>

ESP- und Arduino-Bibliotheken

- [16] R. Scholz, *Syncloop*, Persönliche Mitteilungen

- [17] Knolleary, *PubSubClient*, <https://pubsubclient.knolleary.net/>, Zugriff am: 12. Februar 2025 [Online].
 - [18] ESPWIFI.h, <https://arduino-esp8266.readthedocs.io/en/latest/esp8266wifi/readme.html>
 - [19] EEPROM.h, <https://docs.arduino.cc/learn/built-in-libraries/EEPROM/>
 - [20] M. Carbou, *MycilaEasyDisplay*, <https://github.com/mathieucarbou/MycilaEasyDisplay>, Zugriff: 16. Januar 2025. [Online].
- Python-Bibliotheken**
- [21] Guido van Rossum, Python Software Foundation, *Python*, <https://www.python.org/> Zugriff am 10. Februar 2025 [Online].
 - [22] Python Software Foundation, *json*, <https://docs.python.org/3/library/json.html> Zugriff am: 23. Februar 2025 [Online].
 - [23] Mike Bayer, *SQLAlchemy*, <https://www.sqlalchemy.org> Zugriff am: 11. März 2025 [Online].
 - [24] Sebastián Ramírez (Tiangolo), *FastAPI*, <https://fastapi.tiangolo.com/> Zugriff am: 24. März 2025 [Online].

Abbildungsverzeichnis

1	Gesamtsystem	7
2	Datenbank	32
3	device_element	56
4	Formular zur Erstellung eines Szenarios	57
5	scenario_element	58
6	Formular zur Anmeldenung eines Geräts im Raum	62
7	Gerät in dem Raum	63

Tabellenverzeichnis

1	MQTT-Nachrichten	9
2	Technische Daten des Arduino Uno	10
3	Technische Daten des ESP8266	11
4	Technische Daten des ESP32	12
5	Technische Daten des BME680	13
6	Technische Daten des VCNL4040	13
7	Technische Daten des Raspberry Pi	14

Programmcode

1	Beispiel für asynchrone Abfrage	18
2	ESP: connect_wifi_mqtt()	24
3	ESP: Access Point	25
4	ESP: setup_ap	25

5	ESP: mqtt_connect	26
6	ESP: callback	27
7	M5Stick: Sensoren	28
8	db: __init__	34
9	db: base.py	35
10	models.py: HouseModel	35
11	models.py: UserModel	35
12	models.py: RoomModel	36
13	models.py: DeviceModel	37
14	models.py: RoomDeviceModel	37
15	models.py: ScenarioModel	38
16	queries.py: add_user	39
17	routes.py: get_user_data	43
18	client.py: start_client	47
19	cleint.py: process_message	48
20	script.js: get_user_data	51
21	script.js: mnCreateHouse	52
22	index.html	52
23	script.js: renderLoginForm	53
24	html: renderMainPage	55
25	html: house_element	55
26	html: device_element	56
27	html: renderMyScenariosPage	56
28	script.js: websocket	58
29	style.css	60